

# Hybrid Computing Technology

1

## FPGA-Based Computing Technology

ผศ. อัครินทร์ คุณกิตติ

ASST. PROF. AKHARIN KHUNKITTI

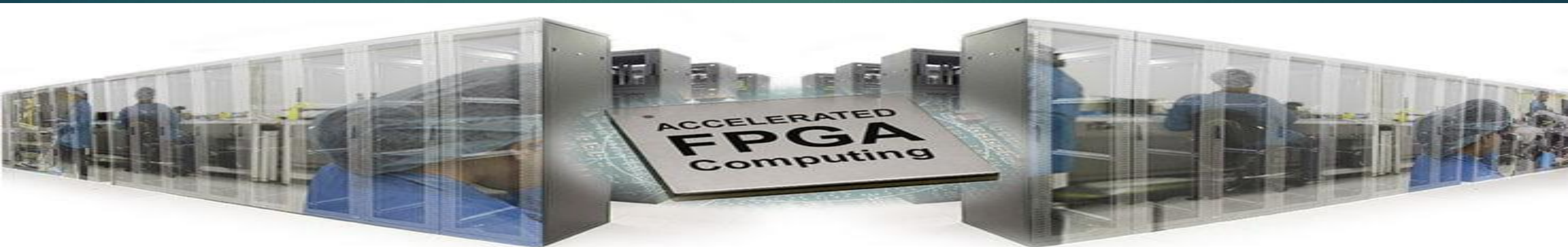
FACULTY OF INFORMATION TECHNOLOGY  
KMITL



# Topics

2

- ▶ Introduction to FPGA
- ▶ FPGA Architecture
- ▶ FPGA Computing and Execution
- ▶ FPGA Code Development
- ▶ Conclusion





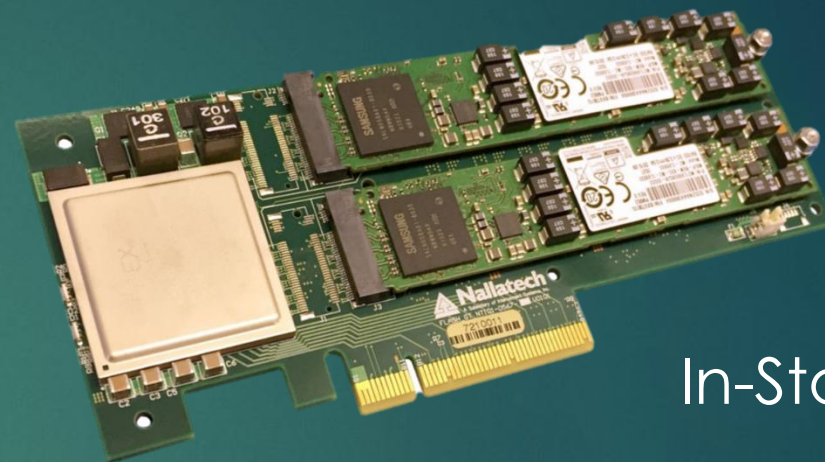


# FPGAs Come In Many Forms

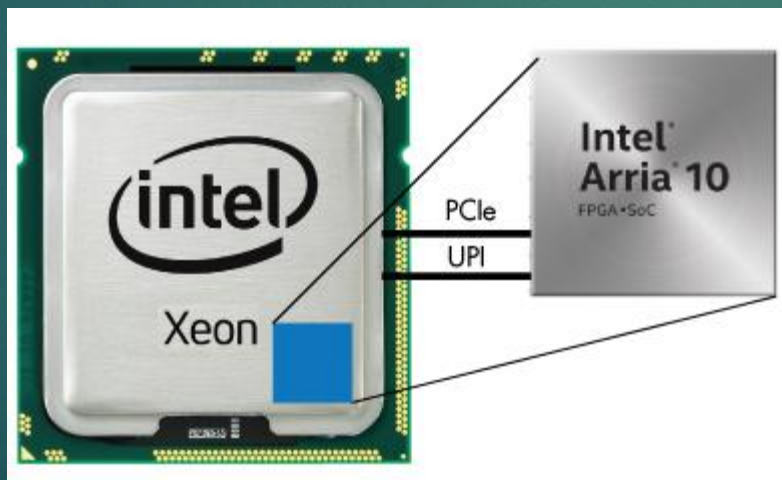
4



PCIe-Attached



In-Storage



CPU Integrated



In-Network



# FPGA History

5

## ► Digital Circuit Design

### ► Truth Table

### ► Logic Equations

#### ► Sum of Products

#### ► Product of Sums

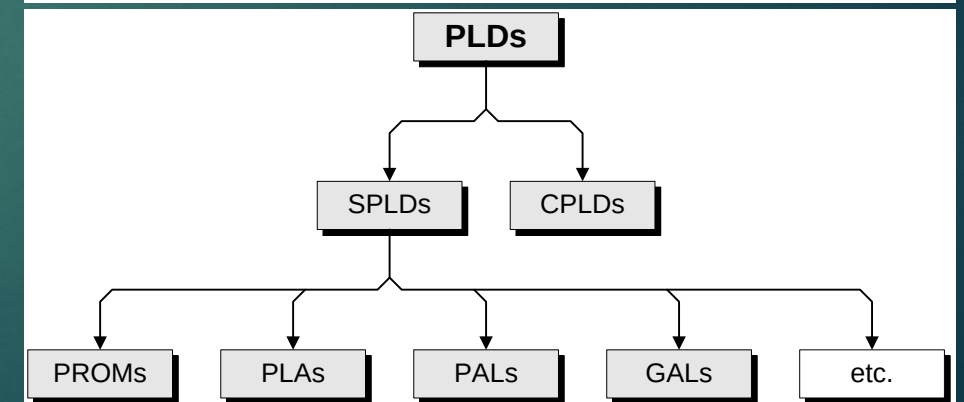
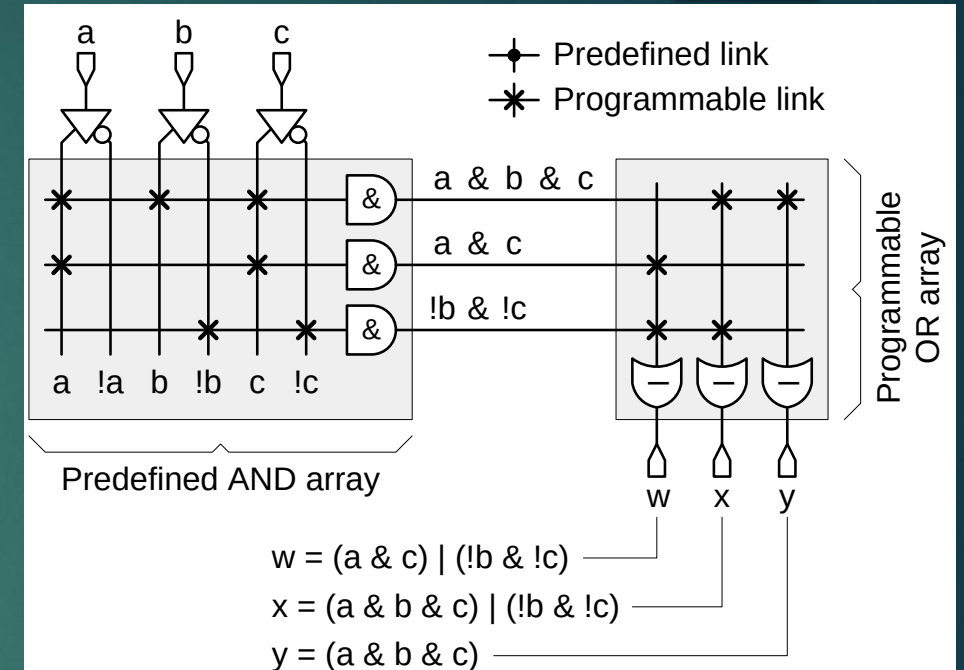
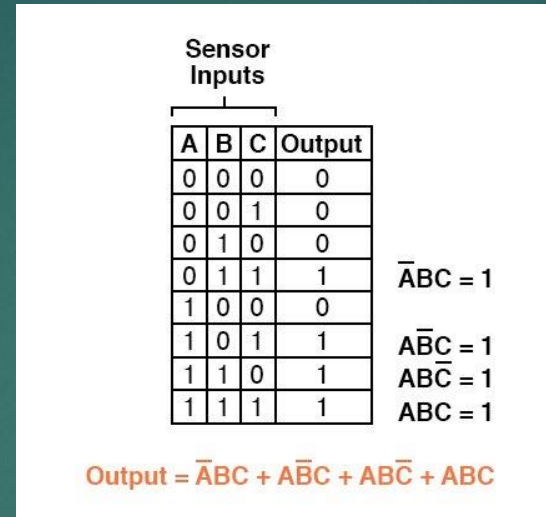
## ► Programmable Logic Devices (PLDs)

### ► Programmable Logic Arrays – about 1970

### ► Programmable Logic Devices – about 1980

### ► Field Programmable Gate Arrays – about 1985

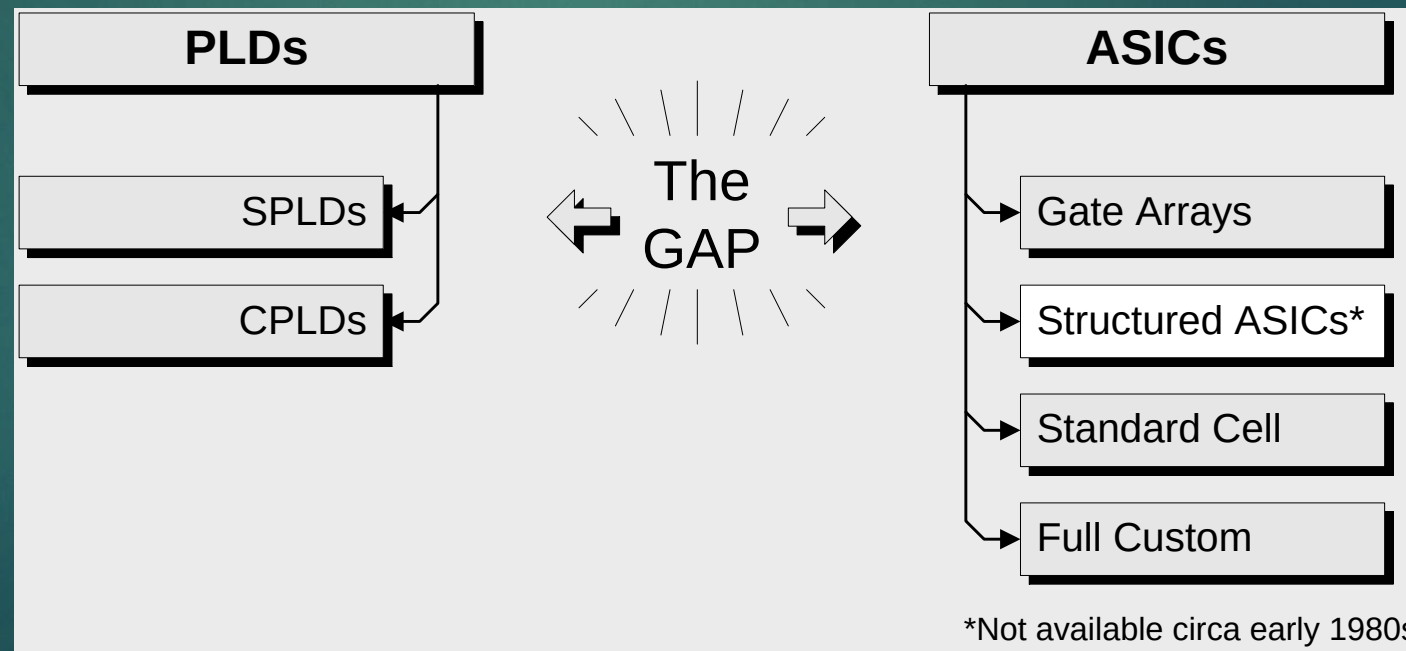
#### ► Xilinx Logic Cell Array



# FPGA and ASIC

6

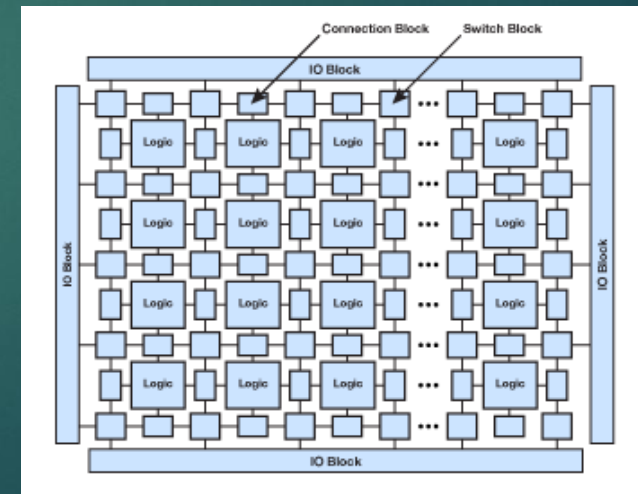
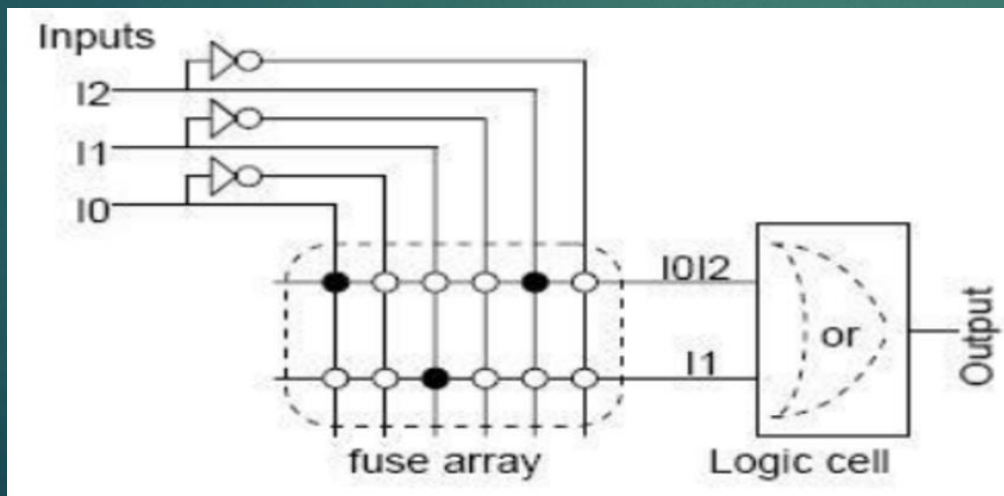
- ▶ FPGAs appeared in the 1980's. Took off in last decade.
- ▶ Bridge gap between simple Programmable Logic and semi custom ASICs (Application Specific Integration Circuits).



# Programmable Logic Device (PLD)

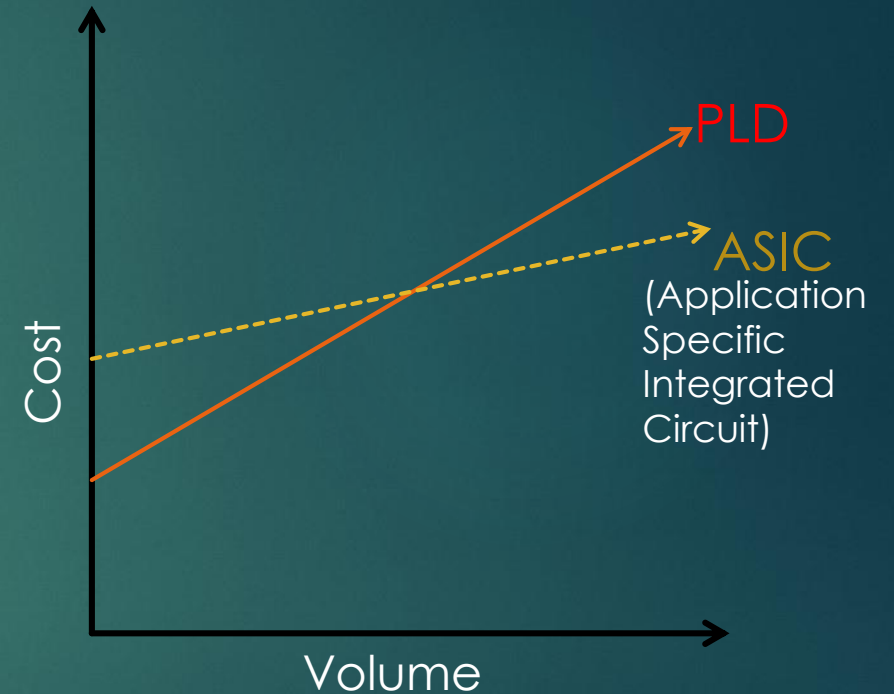
7

- ▶ Also known as “Field Programmable Logic Device (FPLD)”
- ▶ An integrated circuit chip that can be configured by the user to implement different digital hardware.
- ▶ Permits elaborate digital logic designs to be implemented by the user on a single device.
- ▶ Is capable of being erased and reprogrammed with a new design.



# Advantages of PLDs

- ▶ Cost effective in lower volumes
- ▶ Short design time
- ▶ Well suited for academics and prototyping
- ▶ Programmability
- ▶ Re-programmability
  - ▶ PLDs can be reprogrammed without being removed from the circuit board.
- ▶ Low cost of design
- ▶ Immediate hardware implementation



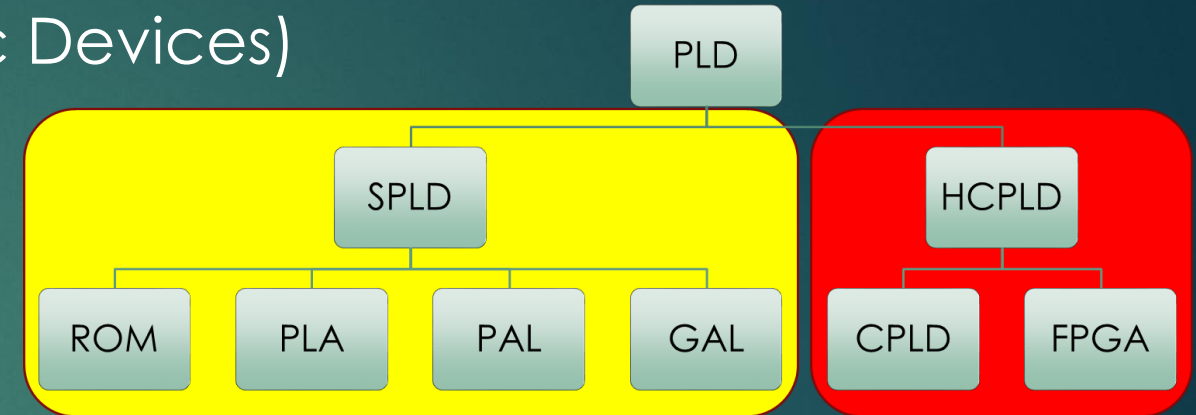


# Types of PLDs

9

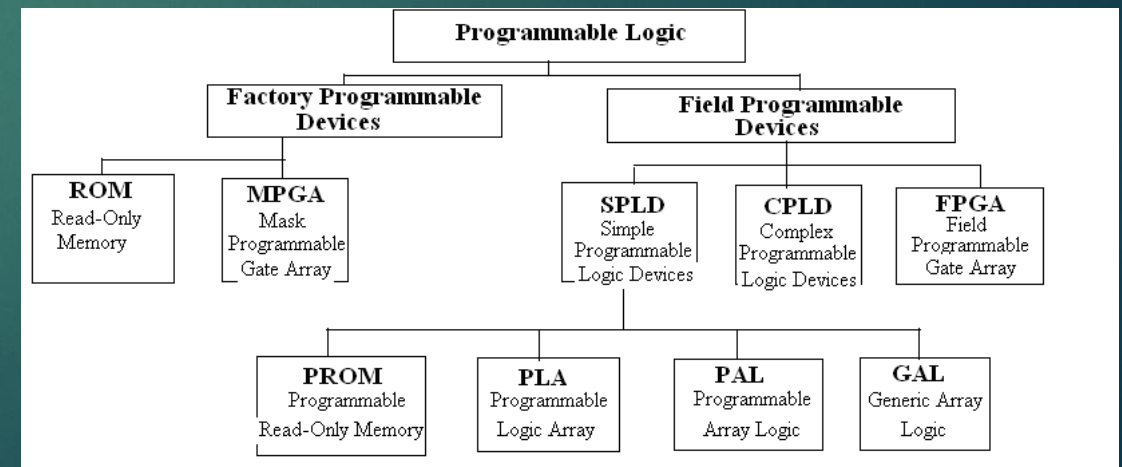
## ► SPLDs (Simple Programmable Logic Devices)

- ROM (Read-Only Memory)
- PLA (Programmable Logic Array)
- PAL (Programmable Array Logic)
- GAL (Generic Array Logic)



## ► HCPLD (High Capacity Programmable Logic Device)

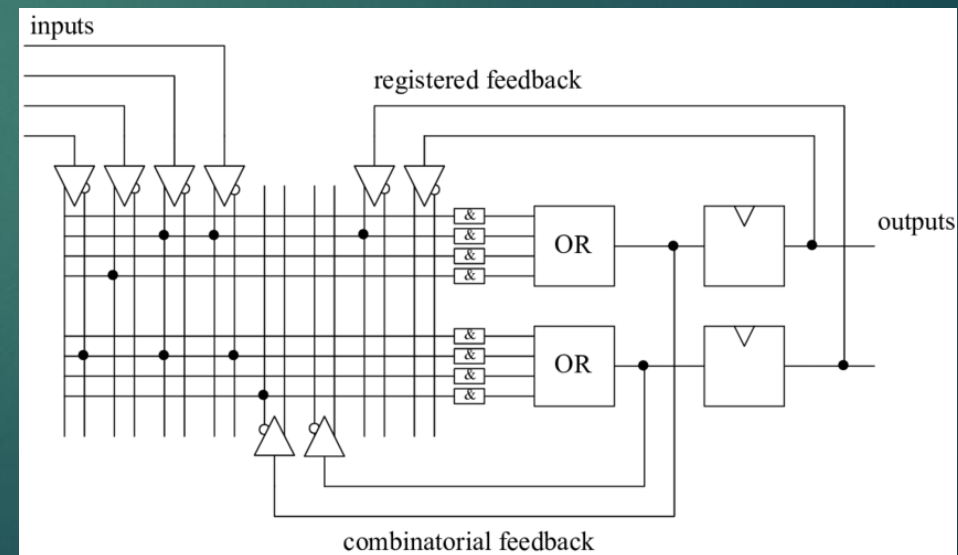
- CPLD (Complex Programmable Logic Device)
- FPGA (Field-Programmable Gate Array)



# SPLDs

10

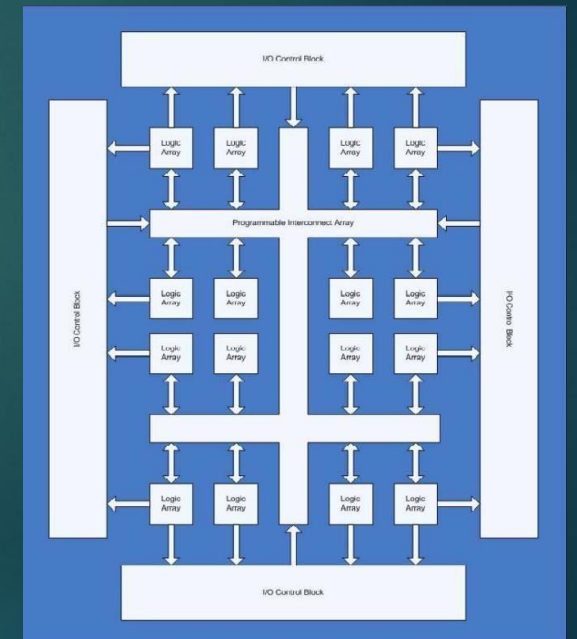
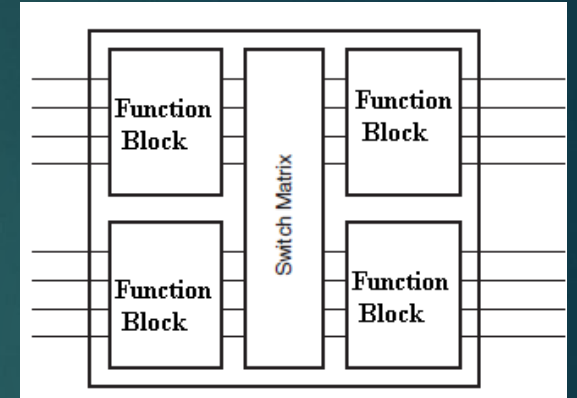
- ▶ In ROM, the input connection matrix is hardwired and the user can only modify the output connection matrix.
- ▶ In PAL and GAL the output connection matrix is also hardwired and the user can modify the input connection matrix.
- ▶ In PLA the user can modify both the input connection matrix and the output connection matrix.



# HCPLDs

11

- ▶ CPLD (Complex Programmable Logic Device)
  - ▶ Lies between PALs and FPGAs in degree of complexity.
  - ▶ Inexpensive
- ▶ FPGA (Field-Programmable Gate Array)
  - ▶ Truly parallel design and operation
  - ▶ Fast turnaround design
  - ▶ Array of logic cells surrounded by programmable I/O blocks





# PLD Configuration

12

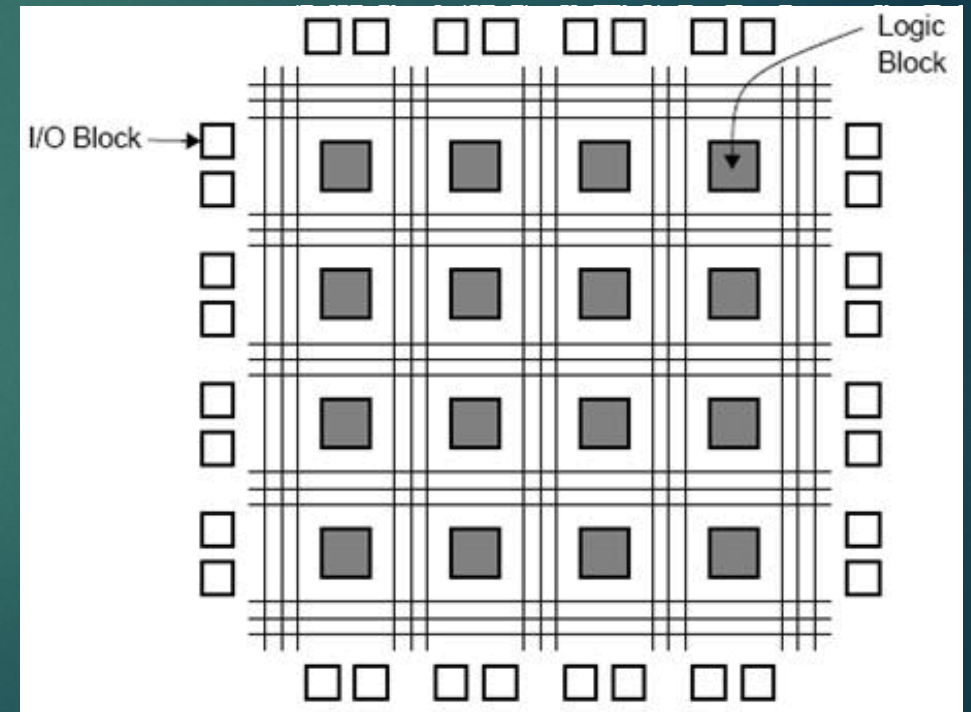
- ▶ Combination of a logic device and memory
- ▶ Memory stores the pattern the PLD was programmed with
  - ▶ EPROM
    - ▶ Non-volatile and reprogrammable
  - ▶ EEPROM
    - ▶ Non-volatile and reprogrammable
  - ▶ Static RAM (SRAM)
    - ▶ Volatile memory
  - ▶ Flash memory
    - ▶ Non-volatile memory
  - ▶ Antifuse
    - ▶ Non-volatile and no re-programmability

| Name     | Re-Programmable          | Volatile |
|----------|--------------------------|----------|
| SRAM     | Yes                      | Yes      |
| FLASH    | Yes                      | No       |
| EPROM    | Yes (out of the circuit) | No       |
| EEPROM   | Yes (in the circuit)     | No       |
| Antifuse | No                       | No       |

# FPGA

13

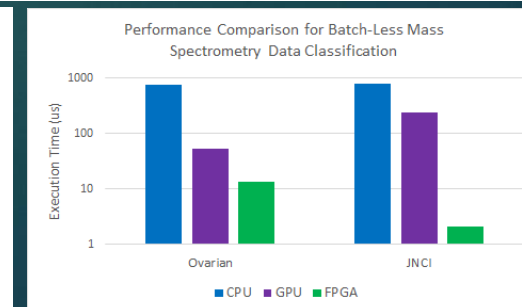
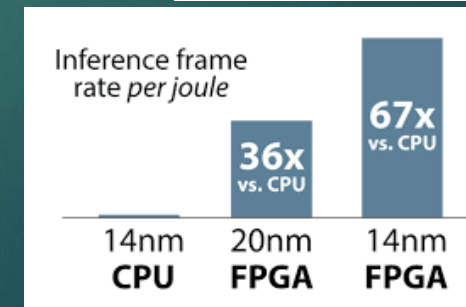
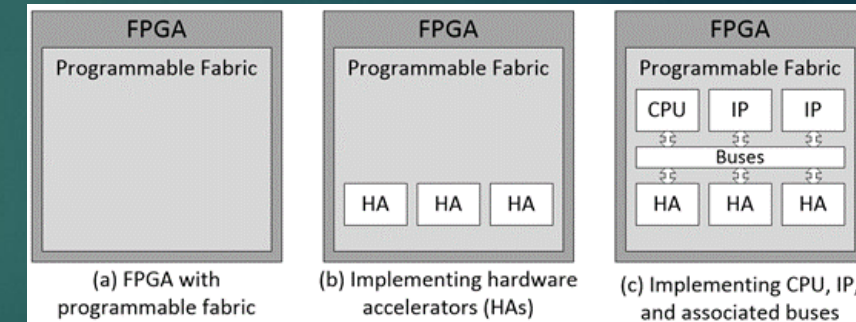
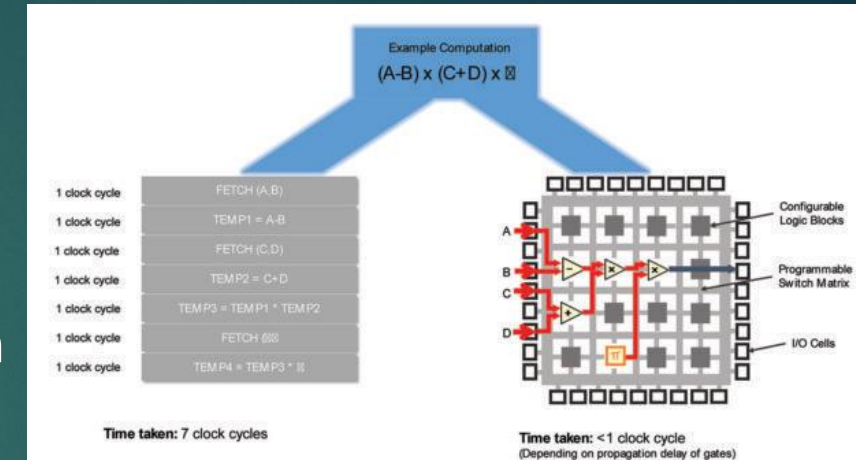
- ▶ Field Programmable Gate Array
- ▶ Field Programmable => Can be programmed by user
- ▶ Gate Array => Set of Logic Gates
- ▶ Compose of
  - ▶ Configurable Logic Block (CLB)
  - ▶ Interconnection or Switch Block
  - ▶ Input / Output Block
  - ▶ Specific Block (Hard Block)



# How Is It Different From CPU/GPUs

14

- ▶ GPU – The other major accelerator
- ▶ CPU/GPU hardware is fixed
  - ▶ “General purpose”
  - ▶ we write programs (sequence of instructions) for them
- ▶ FPGA hardware is not fixed
  - ▶ “Special purpose”
  - ▶ Hardware can be whatever we want
  - ▶ Will our hardware require/support software? Maybe!
- ▶ Optimized hardware is very efficient
  - ▶ GPU-level performance\*\*
  - ▶ 10x power efficiency (300 W vs 30 W)



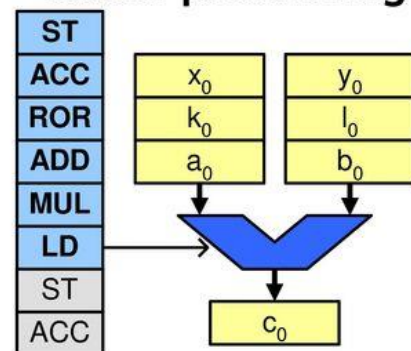


# CPU vs GPU vs FPGA

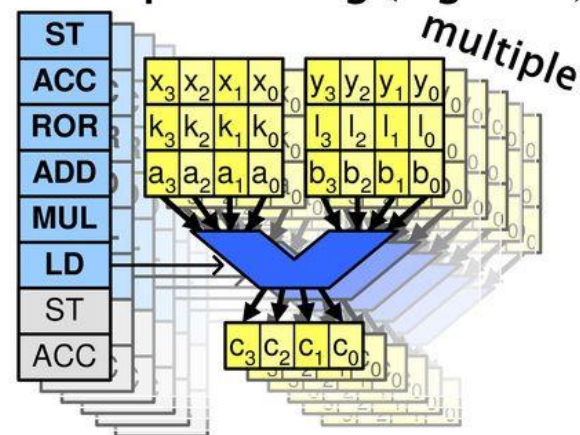
15

## CPUs vs. GPUs vs. FPGAs

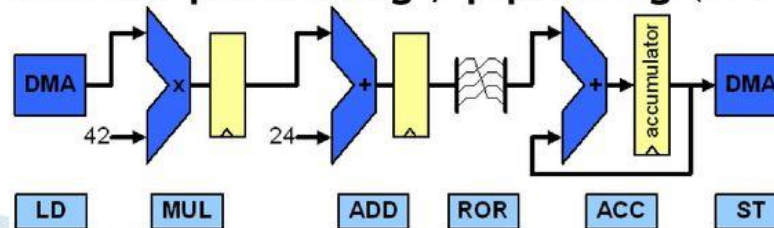
### Scalar processing



### SIMD processing (e.g. GPU)



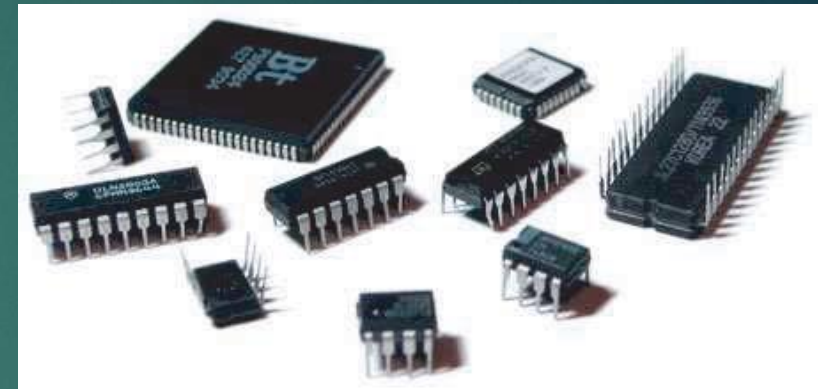
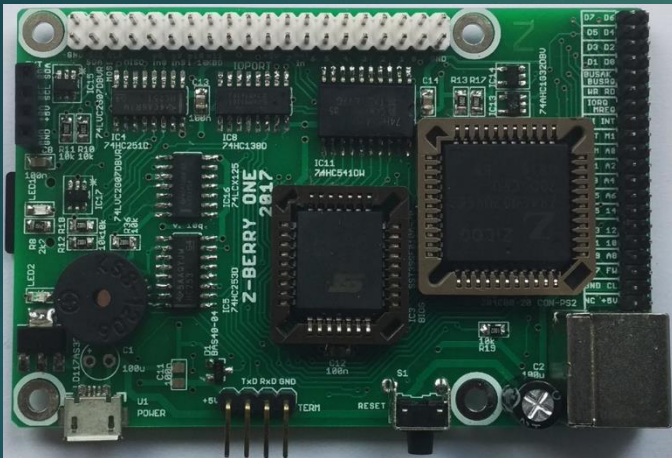
### Dataflow processing / pipelining (FPGAs)



# Analogy

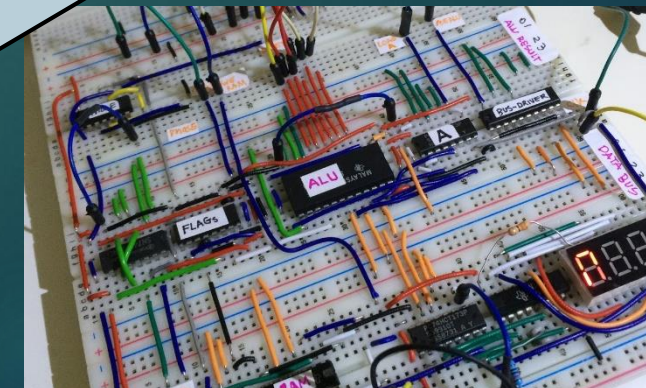
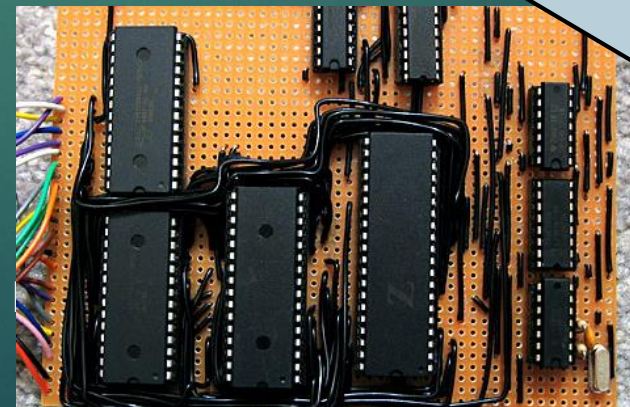
16

CPU/GPU comes with fixed circuits    FPGA gives you a big bag of components



To build whatever

Could be a CPU/GPU!



"The Z-Berry"  
"Experimental Investigations on Radiation Characteristics of  
Copyves.com "Z80 Computer"  
Shadi Soundation: Homebrew 4 bit CPU



# How Is It Different From ASICs

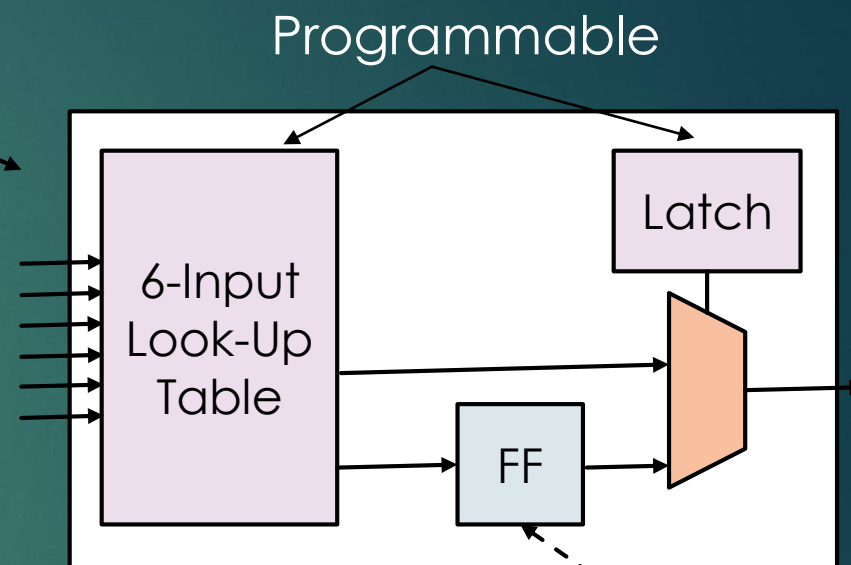
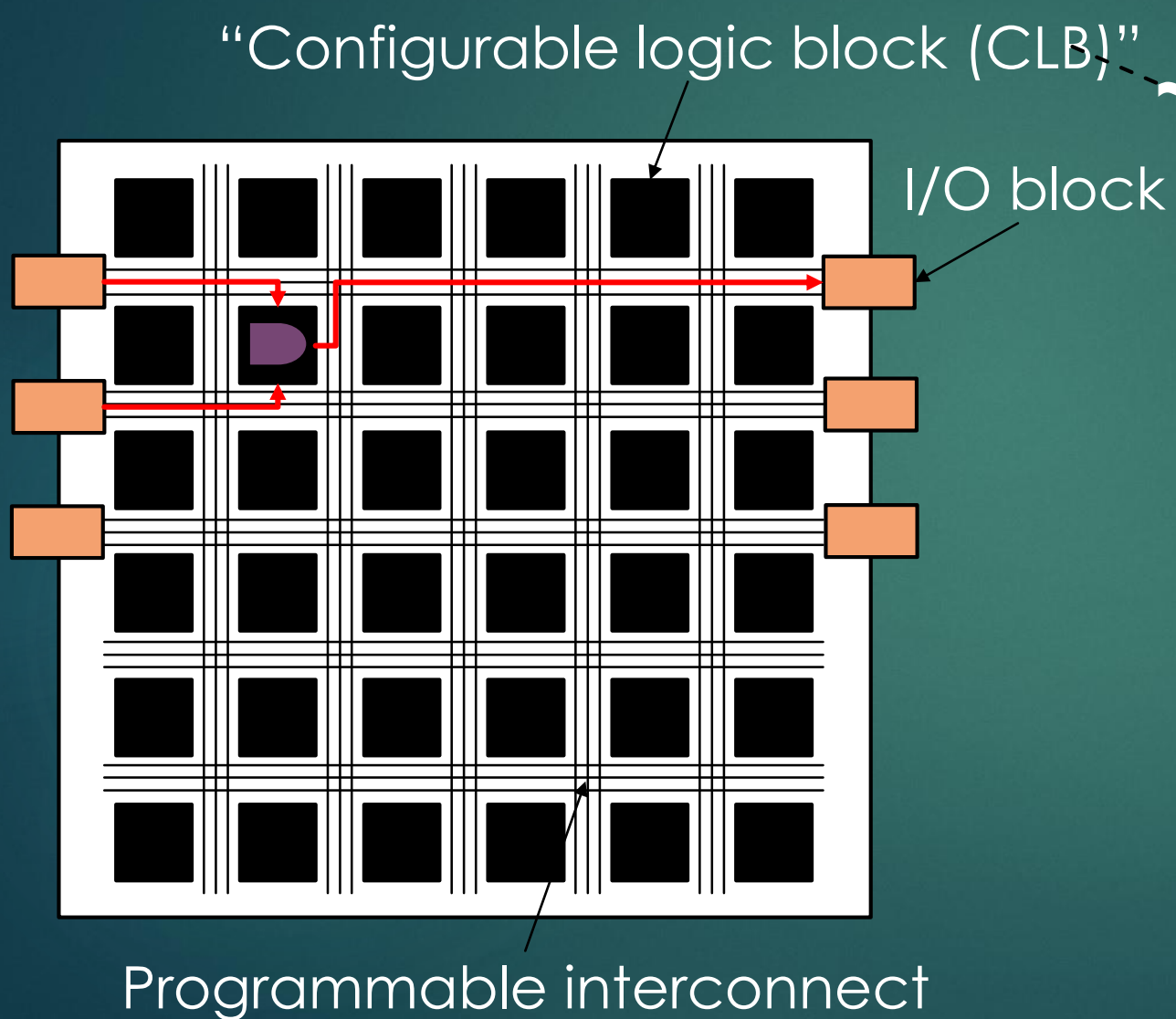
17

- ▶ ASIC (Application-Specific Integrated Circuit)
  - ▶ Special chip purpose-built for an application
  - ▶ E.g., ASIC bitcoin miner, Intel neural network accelerator
  - ▶ Function cannot be changed once expensively built
- ▶ + FPGAs can be **field-programmed**
  - ▶ Function can be changed completely whenever
  - ▶ FPGA fabric **emulates** custom circuits
- ▶ - Emulated circuits are not as efficient as bare-metal
  - ▶ ~10x performance (larger circuits, faster clock)
  - ▶ ~10x power efficiency





# Basic FPGA Architecture



## Ex) 2-LUT for “AND”

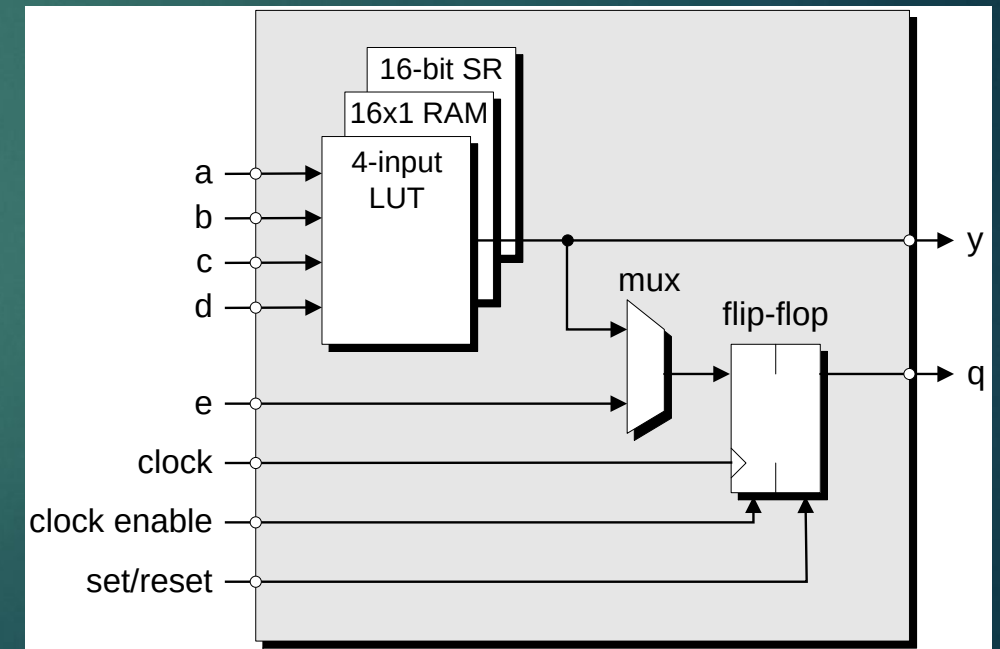
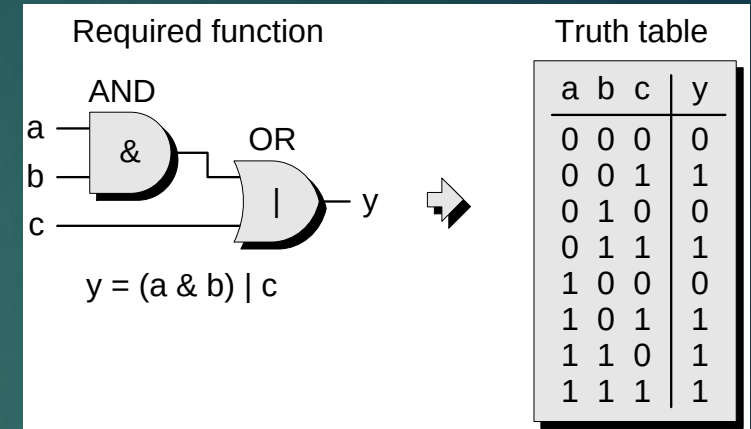
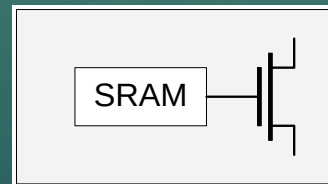
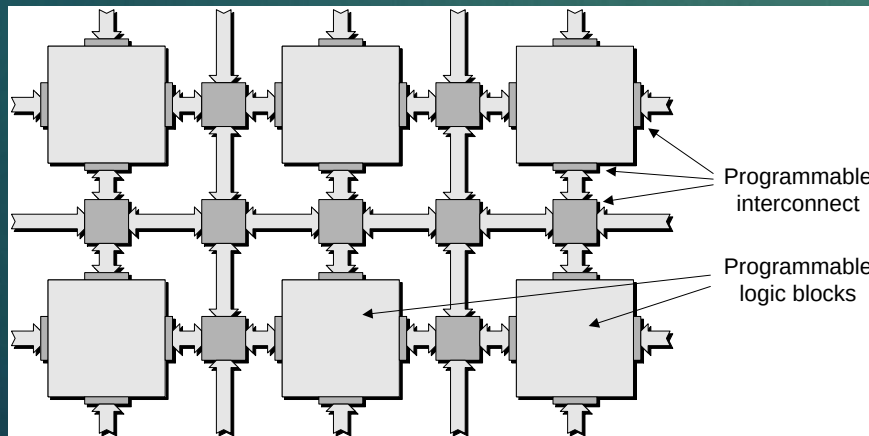
| Input 1 | Input 2 | Output |
|---------|---------|--------|
| 0       | 0       | 0      |
| 0       | 1       | 0      |
| 1       | 0       | 0      |
| 1       | 1       | 1      |

# Sequential circuit construction

# Logic Elements

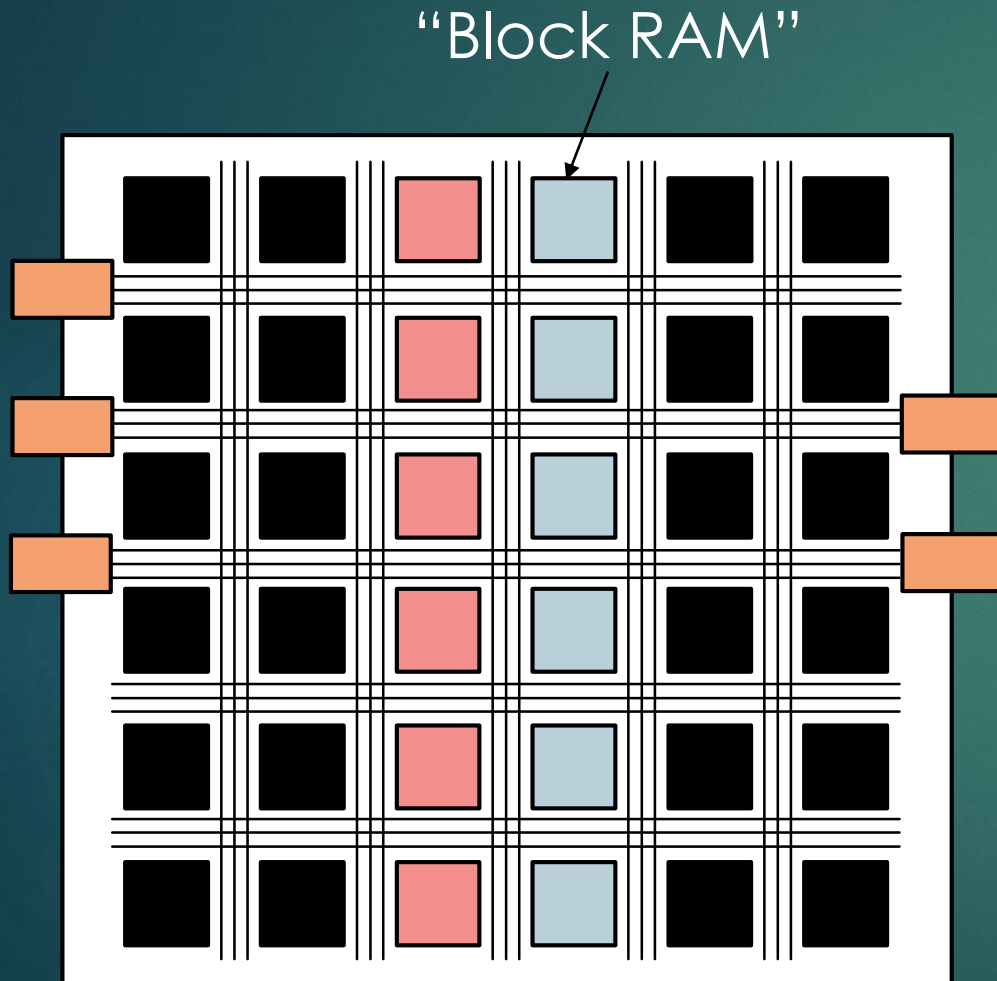
19

- ▶ Lookup Table: LUTs (Combinatorial Logic)
- ▶ Multiplexers
- ▶ Flip-Flops (Clocked Registered Logic)
- ▶ Options configured by SRAM cells



# Basic FPGA Architecture – Block RAM

20



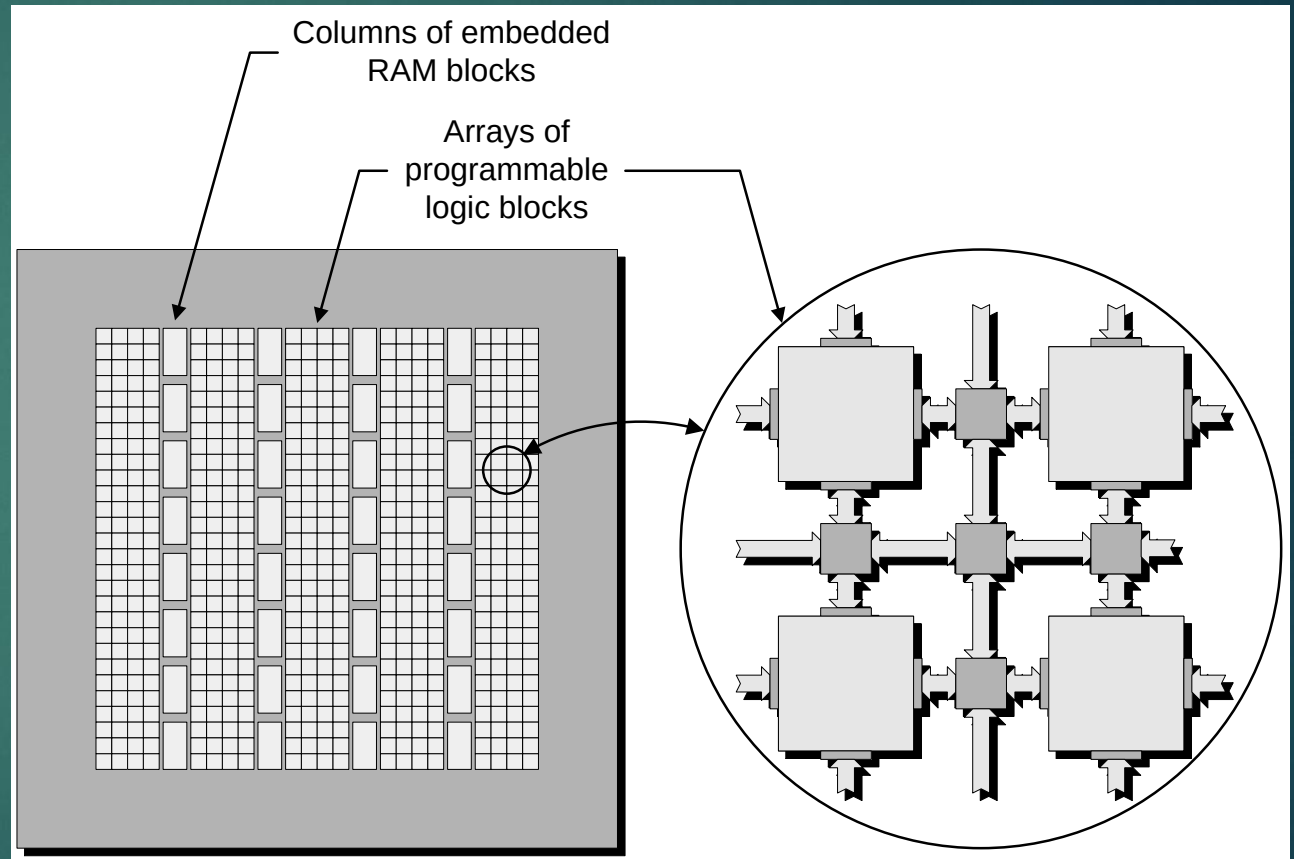
- ▶ CLB can act as flip-flops
  - ▶ (~1 bit/block) – tiny!
- ▶ Some on-chip SRAM provided as blocks
  - ▶ ~18/36 Kbit/block, MBs per chip
  - ▶ Massively parallel access to data → multi-TB/s bandwidth



# Memory

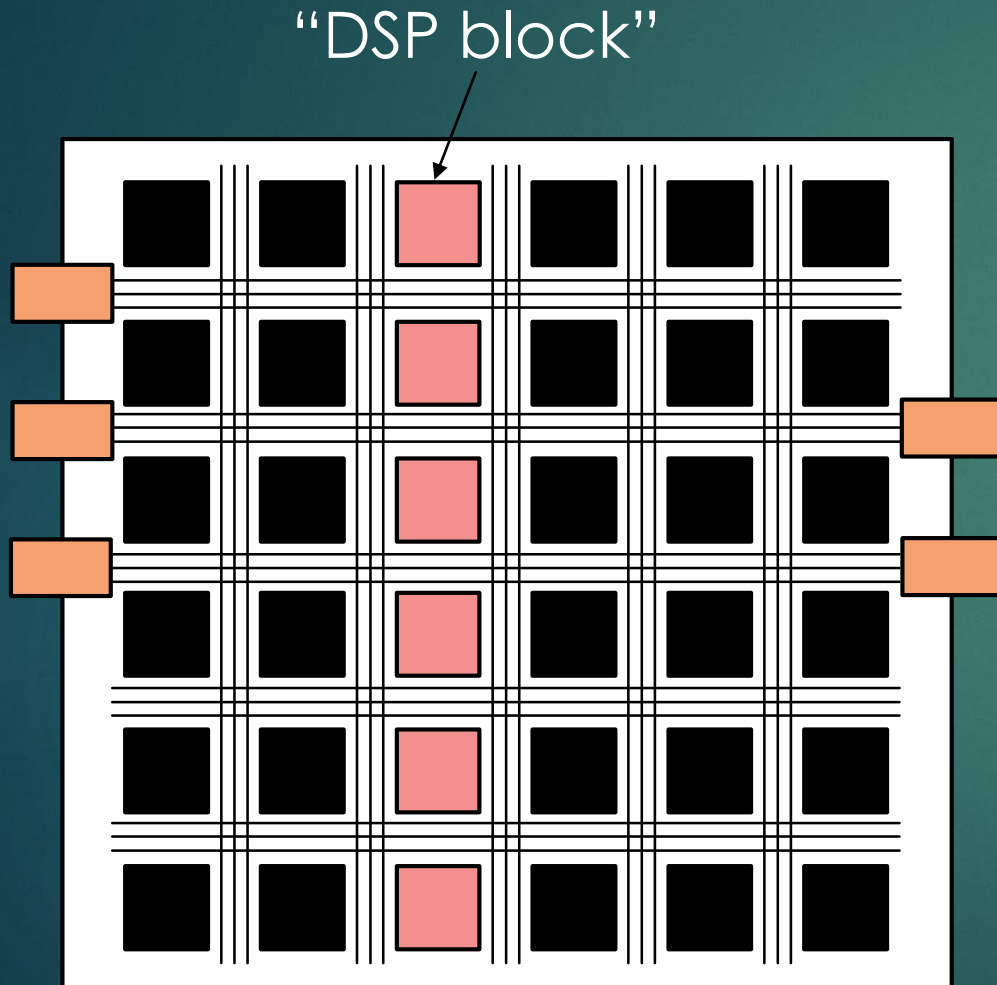
21

- ▶ LUT-RAM
- ▶ SRAM blocks (BRAM)
- ▶ Use for
  - ▶ Data Buffers
  - ▶ FIFOs
  - ▶ Code

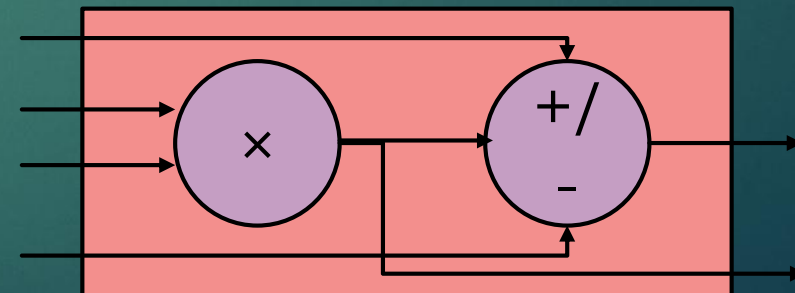


# Basic FPGA Architecture – DSP Blocks

22

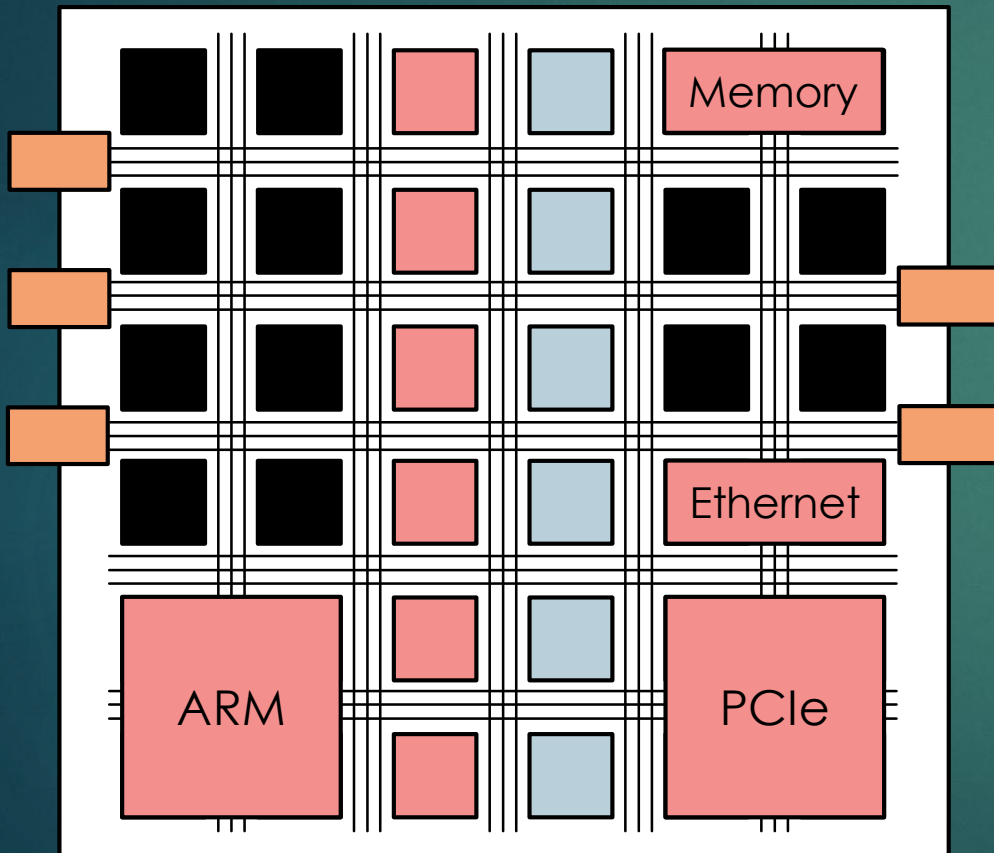


- ▶ CLBs act as gates – Many needed to implement high-level logic
- ▶ Arithmetic operation provided as efficient ALU blocks
  - ▶ “Digital Signal Processing (DSP) blocks”
  - ▶ Each block provides an adder + multiplier



# Basic FPGA Architecture – Hard Cores

23



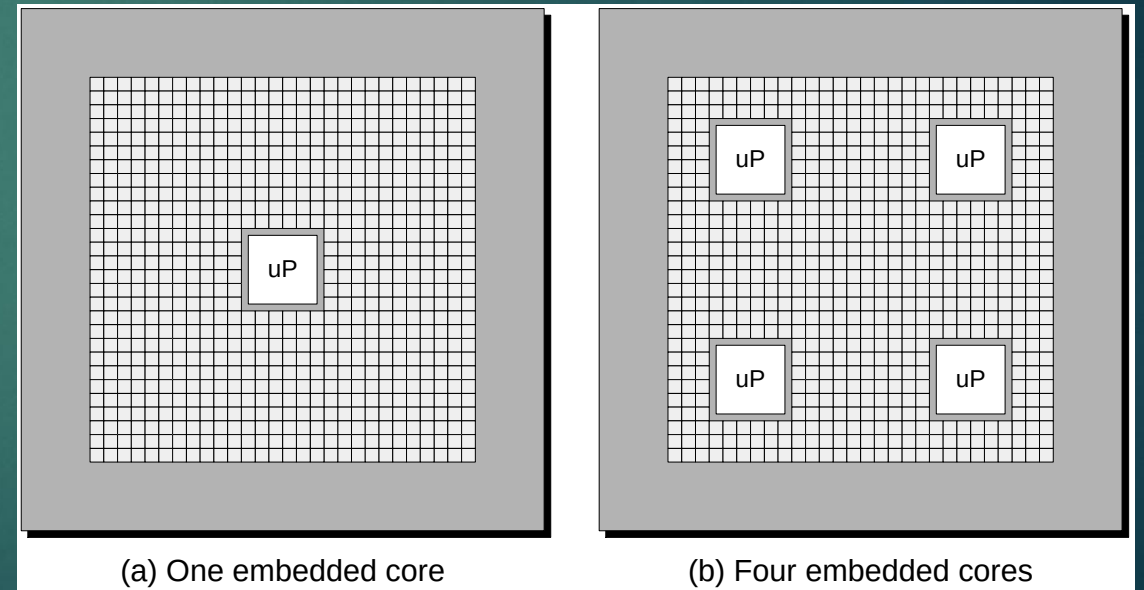
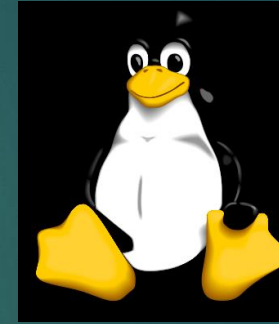
- ▶ Some functions are provided as efficient, non-configurable “hard cores” / “hard blocks”
  - ▶ Multi-core ARM cores (“Zynq” series)
  - ▶ Multi-Gigabit Transceivers
  - ▶ PCIe/Ethernet PHY
  - ▶ Memory controllers
  - ▶ ...



# System on a Chip (SoC)

24

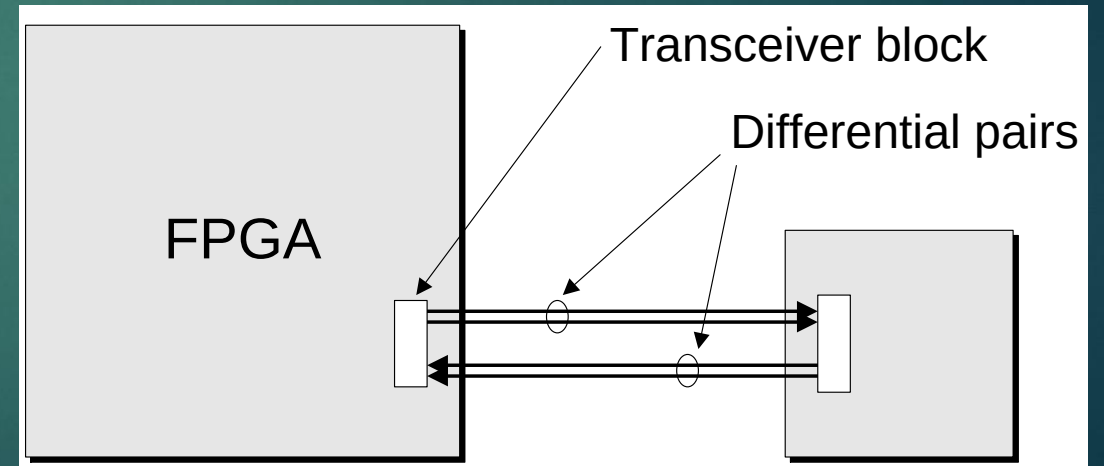
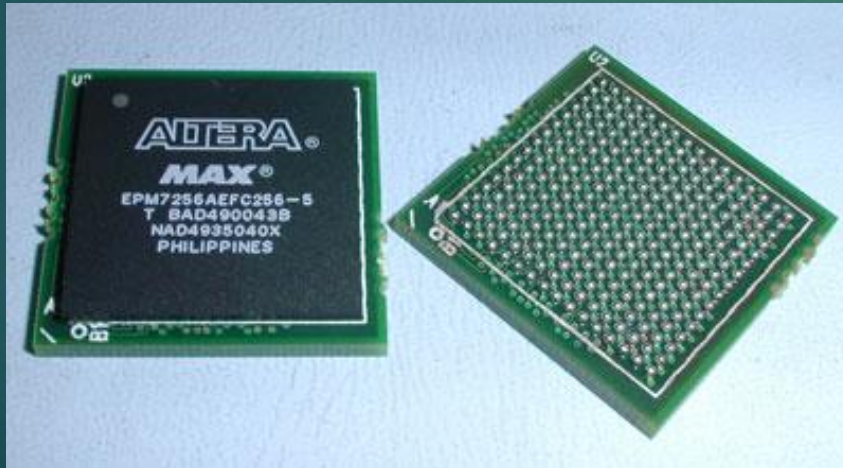
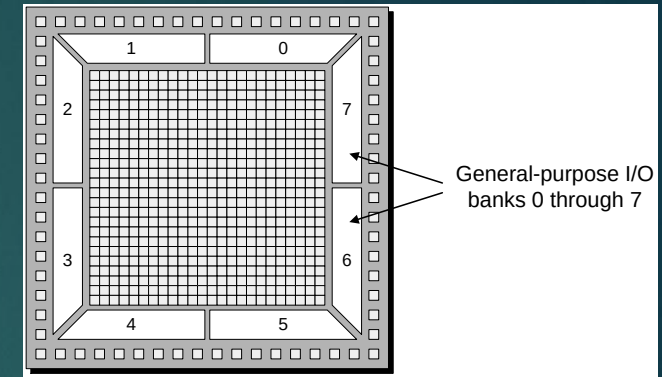
- ▶ Recently Embedded Micro-Processors in Fabric
  - ▶ Hard Cores e.g. RISC PowerPC
  - ▶ Soft Cores / Soft Processors
  - ▶ Peripherals Timers, GPIO
- ▶ Run Operating System e.g. Linux
- ▶ Combine Micro-Processor & Massively Parallel Logic
- ▶ Dual Design Flows
  - ▶ Firmware HDL
  - ▶ Software C



# Input and Output (IO)

25

- ▶ Several hundred of I/O pins
- ▶ All flavours of Logic Levels e.g. LVDS, TTL
- ▶ High Speed Serial Transceivers (up to 10 Gbps)
- ▶ Ethernet MAC Cores



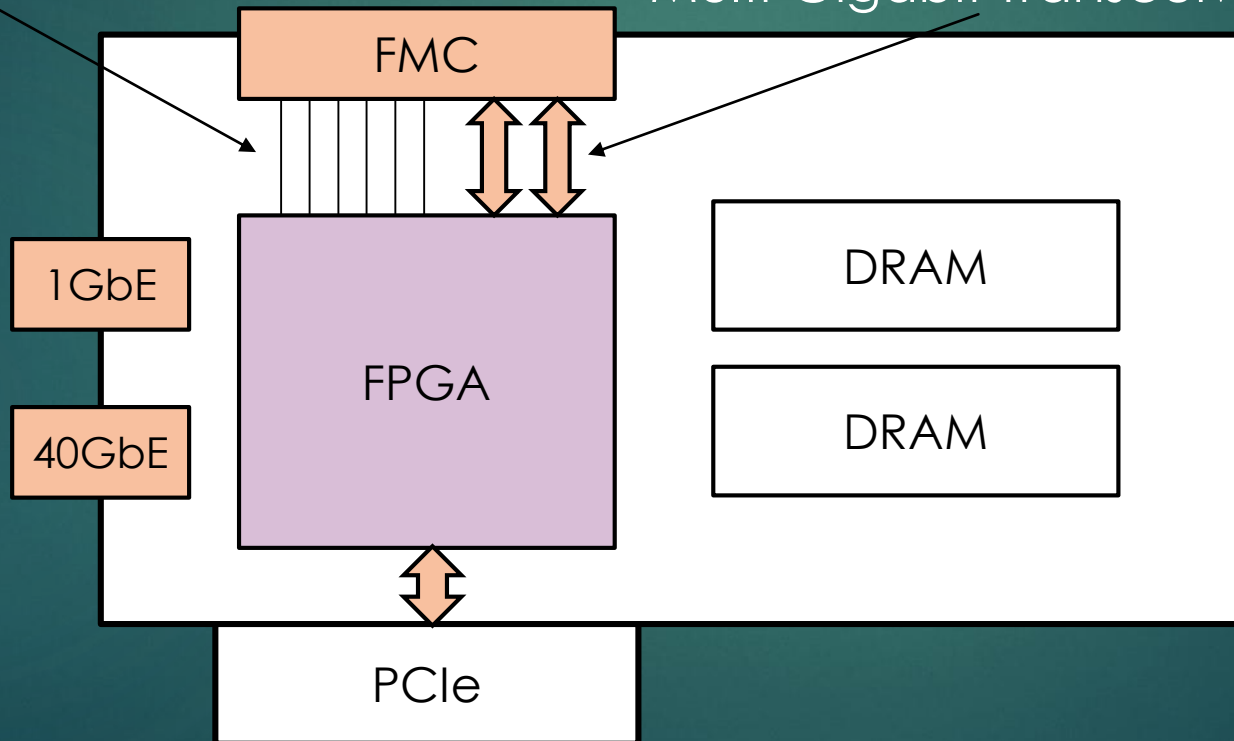
# Example Accelerator Card Architecture

26

- ▶ “FPGA Mezzanine Card” Expansion
- ▶ Network Ports, Memory, Storage, PCIe, ...

General-Purpose I/O Pins

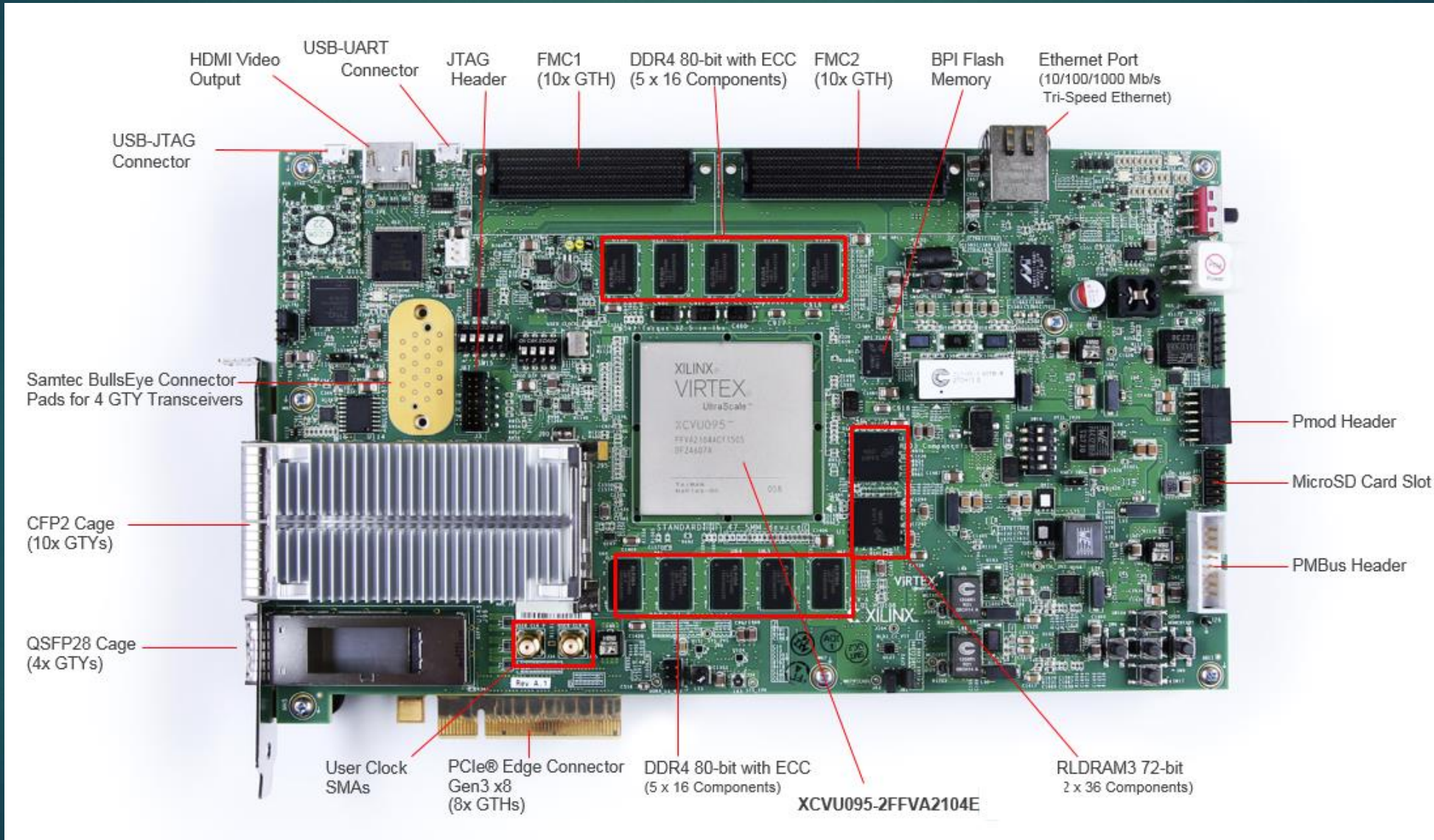
Multi-Gigabit Transceivers





# Example Accelerator Card (VCU108)

27

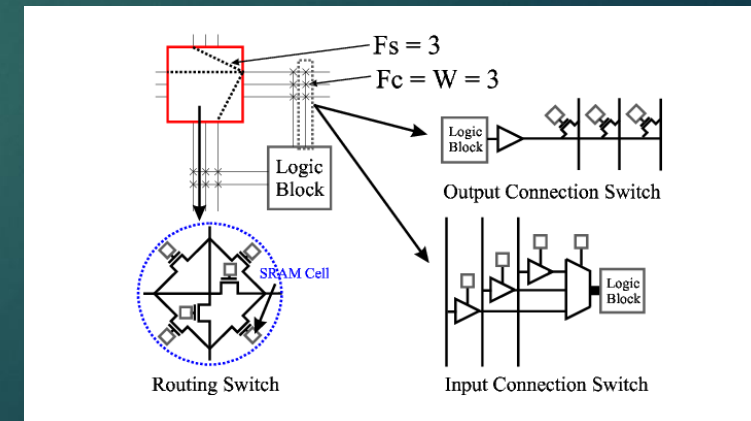
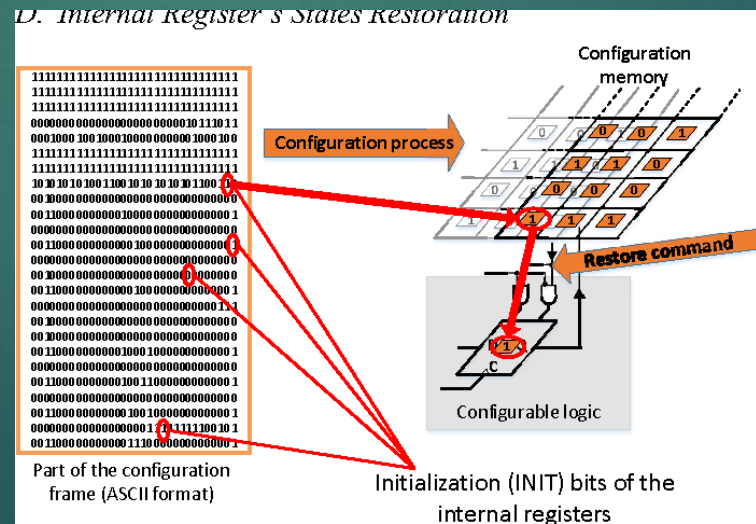
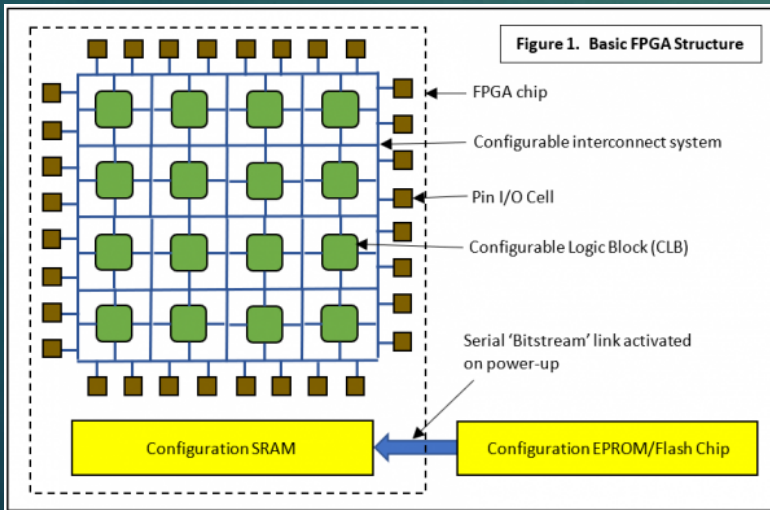
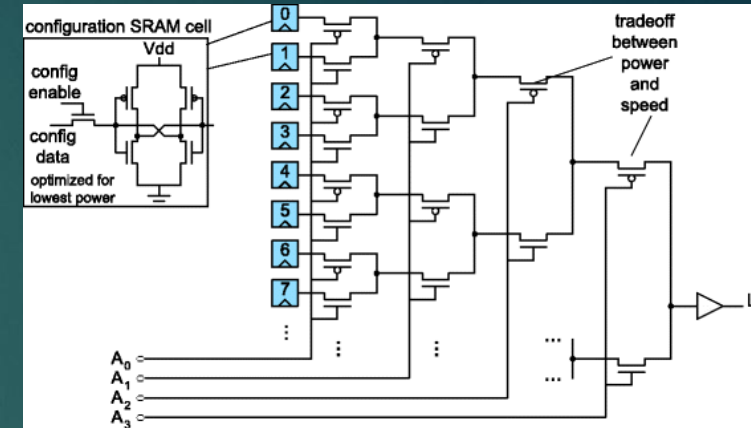




# Programming an FPGA

28

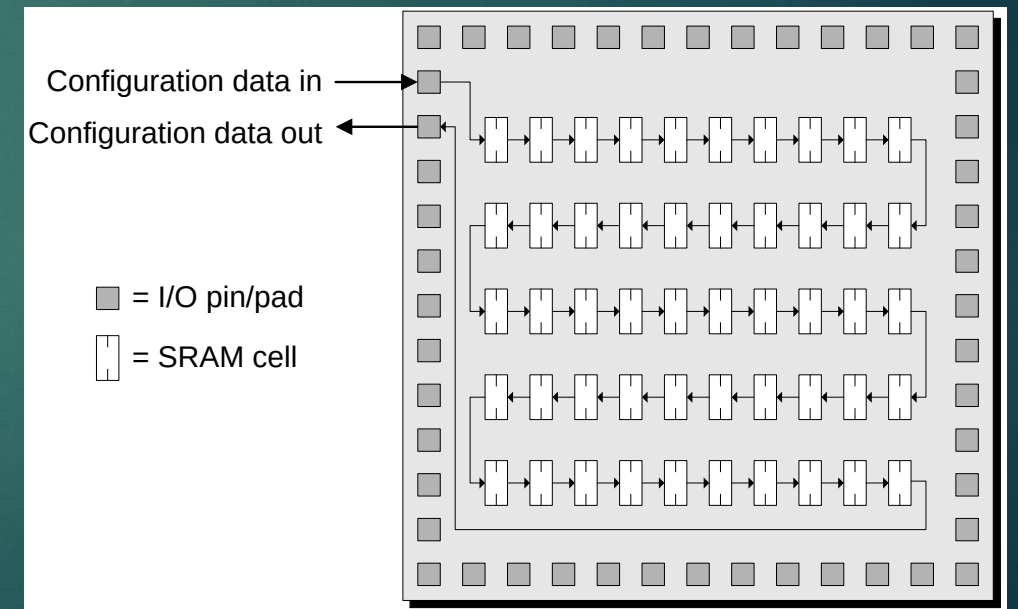
- ▶ Field Programmable Gate Array
  - ▶ Configurable (Programmable) General Logic Blocks
  - ▶ Configurable Interconnects
- ▶ Bit File contains the Configuration Information



# Programming (Configuring) an FPGA

29

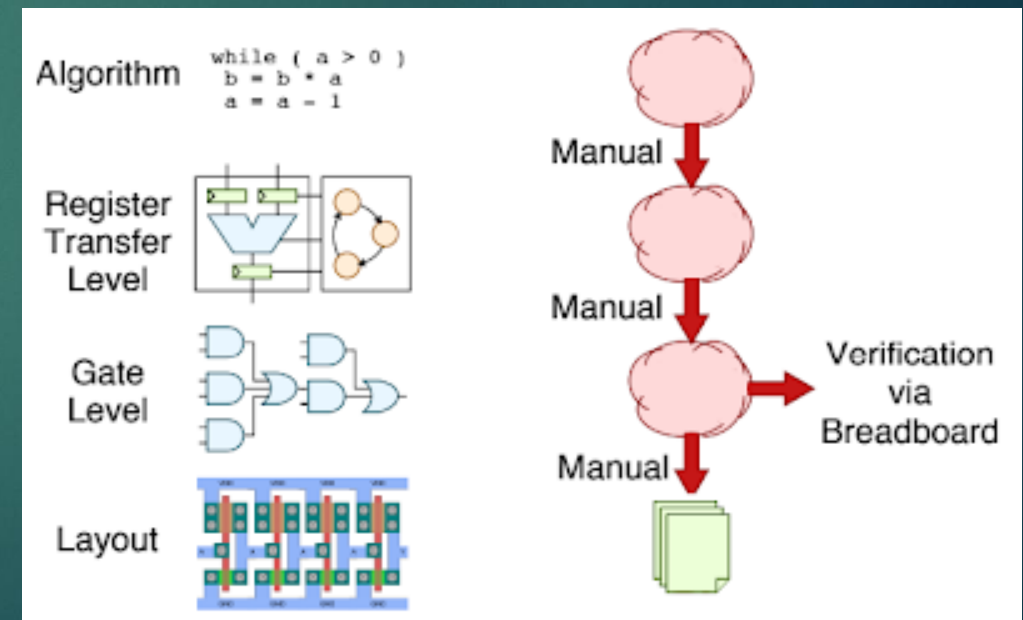
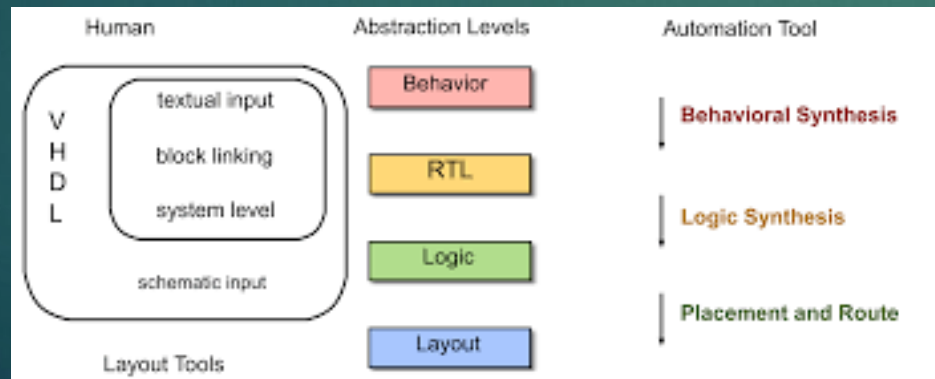
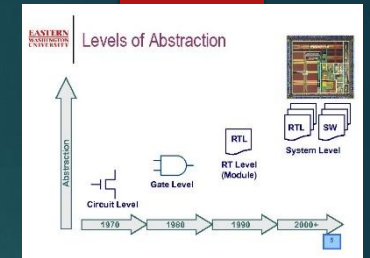
- ▶ SRAM cells holding configuration are Volatile Memory
- ▶ Lose configuration when board power is turned off.
- ▶ Keep Bit Pattern describes the Logic Functions in non-Volatile Memory e.g. ROM or Compact Flash card
- ▶ Reprogramming takes ~ secs
- ▶ Uses JTAG Boundary Scan



# Programming FPGAs

30

- ▶ Languages and tools overlap with ASIC/VLSI design
- ▶ FPGAs for acceleration typically done with either
  - ▶ Hardware Description Languages (HDL): Register-Transfer Level (RTL) languages
  - ▶ High-Level Synthesis: Compiler translates software programming languages to RTL
- ▶ RTL models a circuit using:
  - ▶ Registers (state), and
  - ▶ Combinational logic (computation)

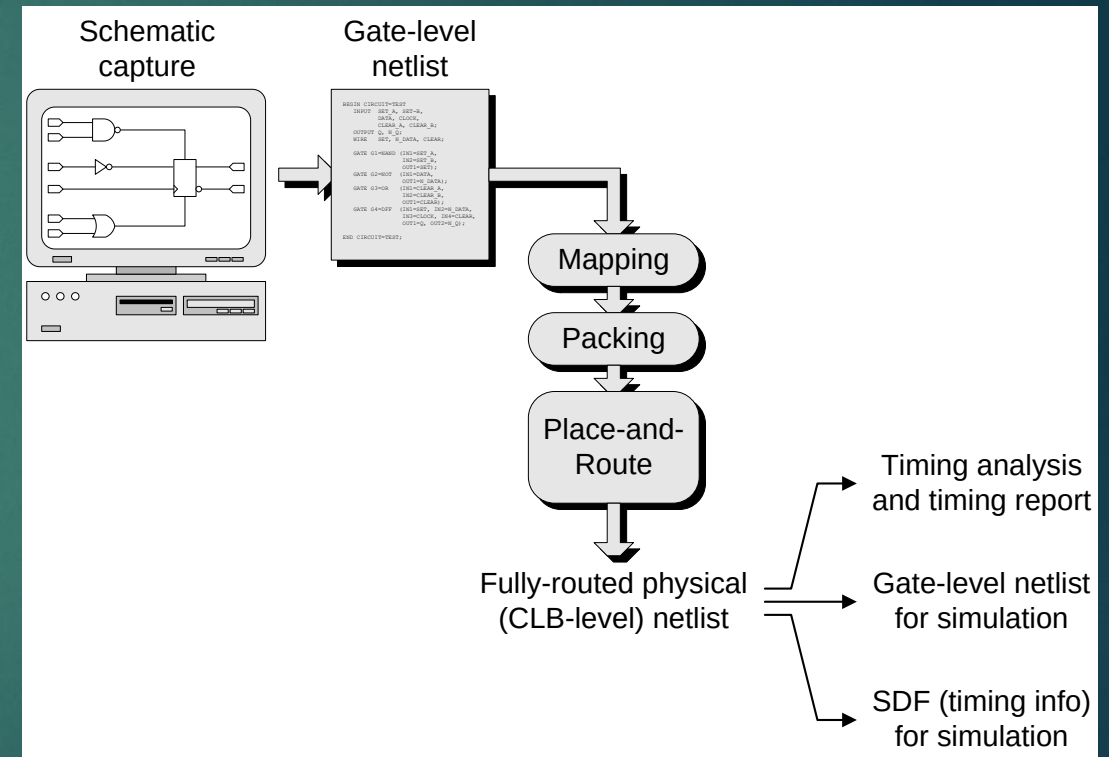




# Design Flows

31

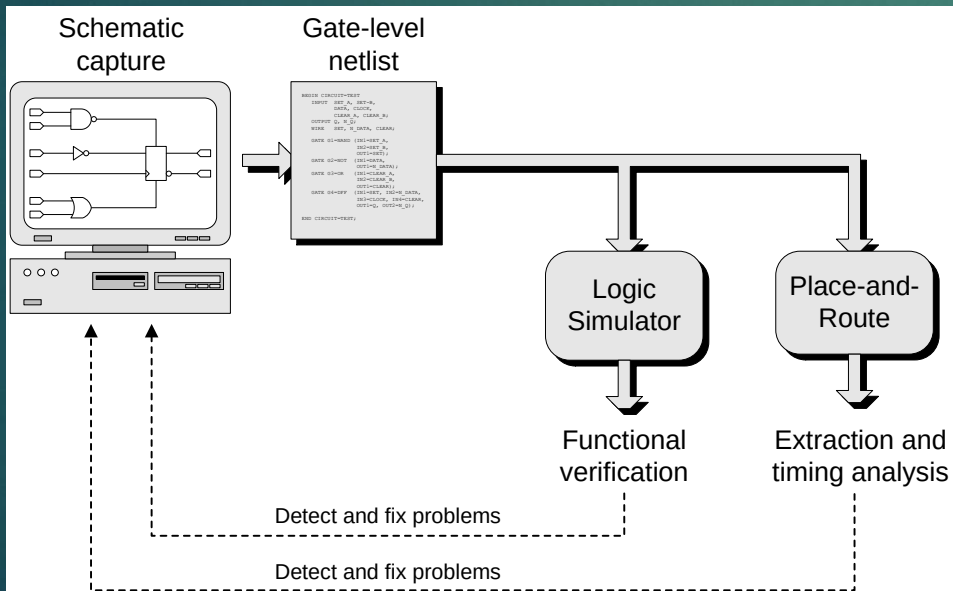
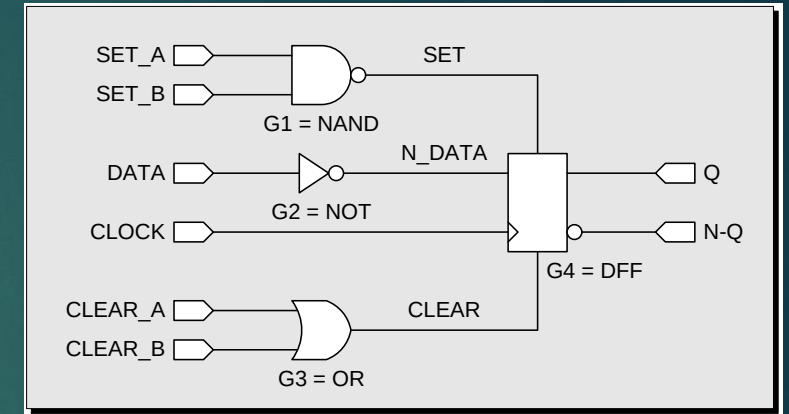
- ▶ High level Description of Logic Design
  - ▶ Schematic
  - ▶ Hardware Description Language
- ▶ Compile into Netlist.
  - ▶ Low (Logic Gates) level description.
- ▶ Target Netlist to FPGA Fabric
  - ▶ Mapping and Packing
  - ▶ Placing and Routing
- ▶ Tools Generate the Bit File
- ▶ Simulation
- ▶ Timing Analysis



# Design Flows

32

- ▶ Schematic Capture of Logic Design.
- ▶ Useful at Top level.
- ▶ Create Netlist. Text file with signal connections.



```
BEGIN CIRCUIT=TEST
  INPUT  SET_A, SET_B, DATA, CLOCK, CLEAR_A, CLEAR_B;
  OUTPUT Q, N_Q;
  WIRE   SET, N_DATA, CLEAR;

  GATE G1=NAND (IN1=SET_A, IN2=SET_B, OUT1=SET);
  GATE G2=NOT  (IN1=DATA, OUT1=N_DATA);
  GATE G3=OR   (IN1=CLEAR_A, IN2=CLEAR_B, OUT1=CLEAR);
  GATE G4=DFF  (IN1=SET, IN2=N_DATA, IN3=CLOCK,
               IN4=CLEAR, OUT1=Q, OUT2=N_Q);

END CIRCUIT=TEST;
```

# Hardware Description Language

33

- ▶ Software programming languages: Describes process
- ▶ Hardware description languages: Describes structure

```
std::queue<float> input_queue;  
std::queue<float> output_queue;  
float factor;
```

Exists in memory

```
while (true) {  
    if ( !input_queue.empty() ) {  
        ret = input_queue.front() * factor;  
        output_queue.push(ret)  
        input_queue.pop();  
    }  
}
```

Instructions For CPU

```
FIFO#(Float) input_queue <- mkFIFO;  
FIFO#(Float) output_queue <- mkFIFO;  
Reg#(Float) factor <- mkReg;  
FloatMultIfc mult <- mkFloatMult;
```

Exists on chip

```
rule in;  
    mult.enq(factor, input_queue.first);  
    input_queue.deq;  
endrule  
rule out;  
    ret <- mult.result;  
    output_queue.enq(ret);  
endrule
```

Creates circuits



# Major Hardware Description Languages

34

- ▶ Verilog: Most widely used in industry
  - ▶ Relatively low-level language supported by everyone
- ▶ VHDL (VHSIC HDL): Widely used in industry
  - ▶ Relatively low-level language supported by everyone
- ▶ Chisel – Compiles to Verilog
  - ▶ Relatively high-level language from Berkeley
  - ▶ Embedded in the Scala programming language
  - ▶ Prominently used in RISC-V development (Rocket core, etc)
- ▶ Bluespec – Compiles to Verilog
  - ▶ Relatively high-level language from MIT
  - ▶ Supports types, interfaces, etc
  - ▶ Also active RISC-V development (Piccolo, etc)

| Verilog                 | VHDL               |
|-------------------------|--------------------|
| ASIC Designs            | FPGA Designs       |
| Weakly Typed            | Strongly Typed     |
| Low Verbosity           | High Verbosity     |
| Partially Deterministic | Very Deterministic |
| More “C” like           | Non “C” like       |

```
clk:      in std_logic;
rst:      in std_logic;

-- inputs
data_in:  in std_logic_vector (DATA_W-1);
load:     in std_logic;
en:       in std_logic;

data_out: out std_logic_vector (DATA_W-1);
done:     out std_logic;
end component;

signal load_vec : std_logic_vector (TIMERS-1 down to 0);

begin
  demux: process(load_sel, load)
  begin
    -- set all to 0
    load_vec <= (others => '0');
    -- Set load signal to the addressed timer
    load_vec(to_integer(unsigned(load_sel)))
    end process;
end
```

**VHDL**

```
17 localparam S2 = 2'b11;
18 localparam S3 = 2'b10;
19
20 reg [1:0] pState, nState;
21
22 always @ (pState, start, stop, increment)
23 begin
24   case (pState)
25   S0:begin
26     if (start == 1'b0 && increment == 1'b0)
27       nState = S0;
28     else if (increment == 1'b1 && start == 1'b0)
29       nState = S2;
30     else
31       nState = S1;
32   end
33   S1:
34   begin
35     if (stop == 1'b1)
36       nState = S0;
37     else
38       nState = S1;
39   end
40   S2:
41     nState = S3;
42   S3:
43   begin
44     if (increment == 1'b1)
45       nState = S3;
46   end
47 end
```

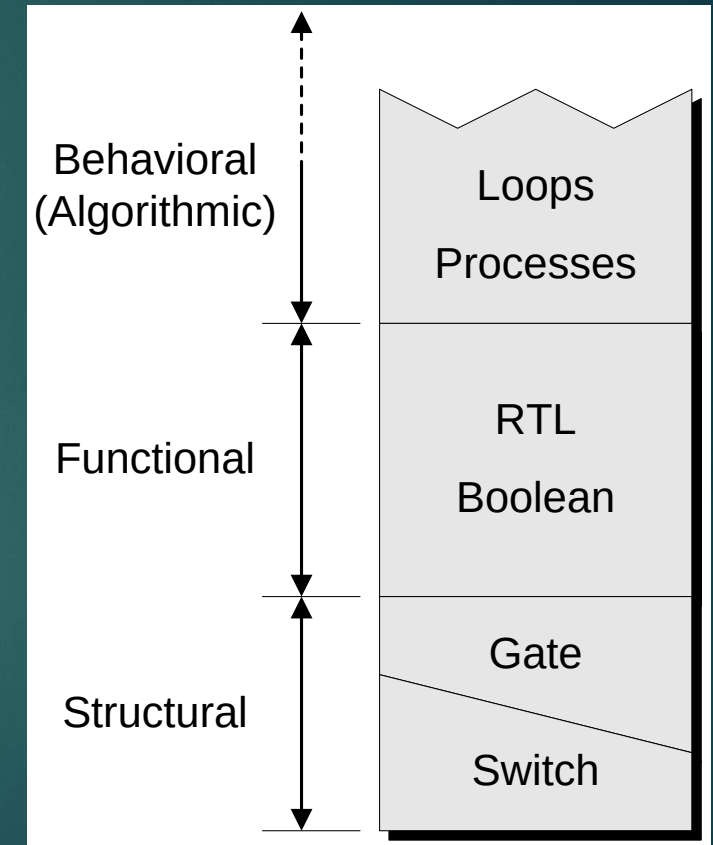
**Verilog**

**VS**

# Hardware Description Languages

35

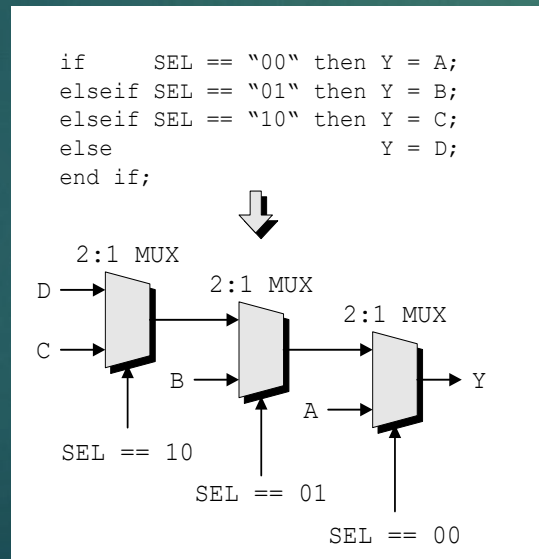
- ▶ Behavioural / Register Transfer Level Description
  - ▶ Program Statements. Loops. If Statements ...etc
- ▶ Describing Mixture of Combinatorial and Sequential Logic and Signals between.
- ▶ Engineers call it Firmware
- ▶ VHDL (VHSIC Hardware Description Language)
  - ▶ Very High Speed Integrated Circuit
- ▶ VERILOG (US)
- ▶ Synthesis (Compilation)
- ▶ Generate Netlist



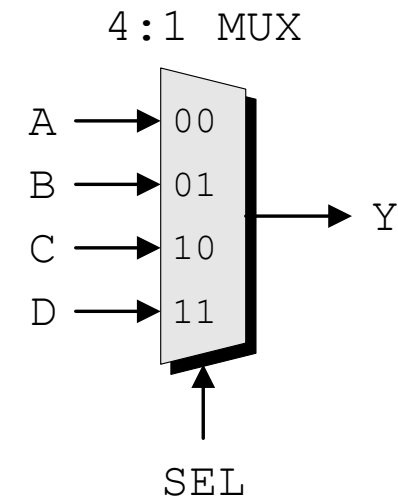
# Hardware Description Languages

36

- ▶ View Logic as collection of Processes operating in Parallel
- ▶ Language Constructs for Multiplexers, FlipFlops ...etc
- ▶ Restrictive set of RTL for Synthesis
- ▶ Synthesis Tools recognise certain code constructs and generate appropriate logic



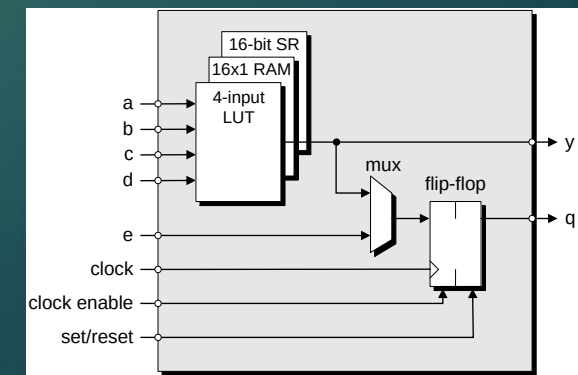
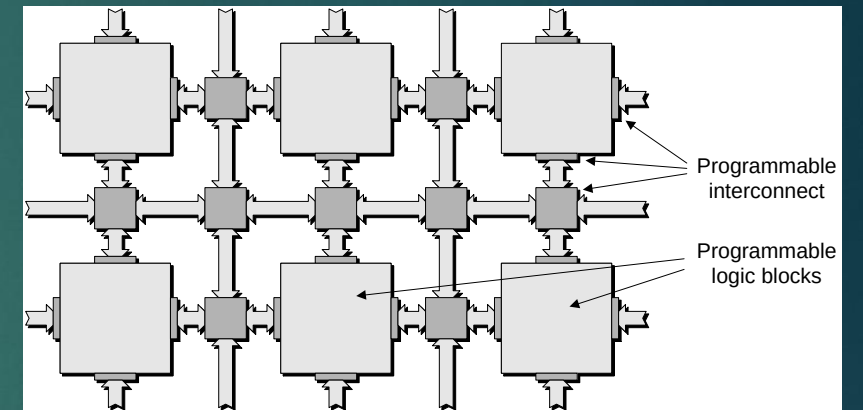
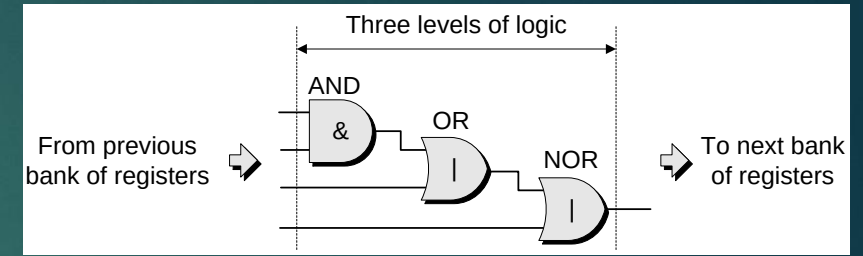
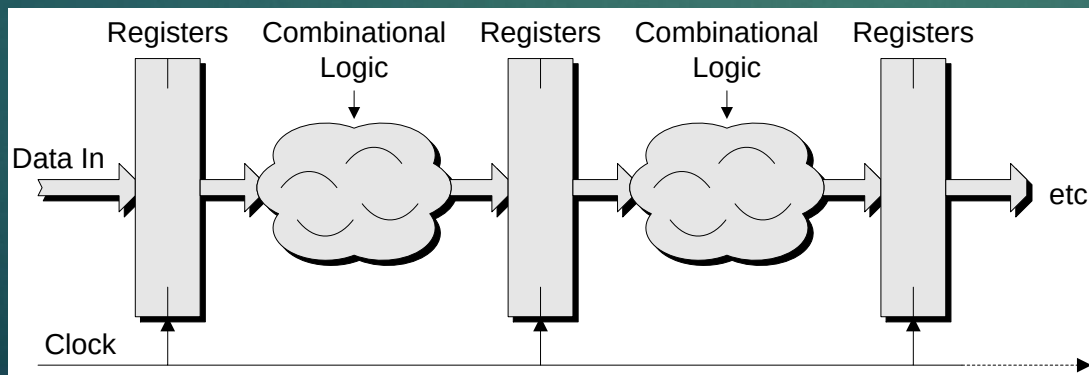
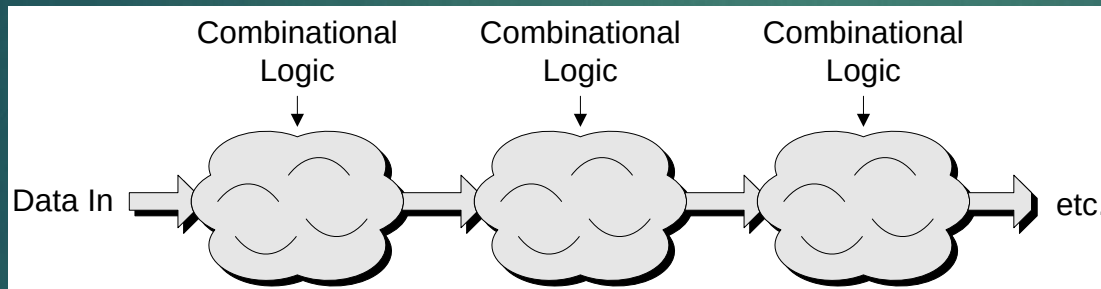
```
case SEL of;
  "00": Y = A;
  "01": Y = B;
  "10": Y = C;
otherwise: Y = D;
end case;
```



# FPGA Design Synchronous Logic

37

- ▶ Pipelined. Clocked Logic.
- ▶ Combinational and Sequential Logic.
- ▶ Register Transfer Level Logic.

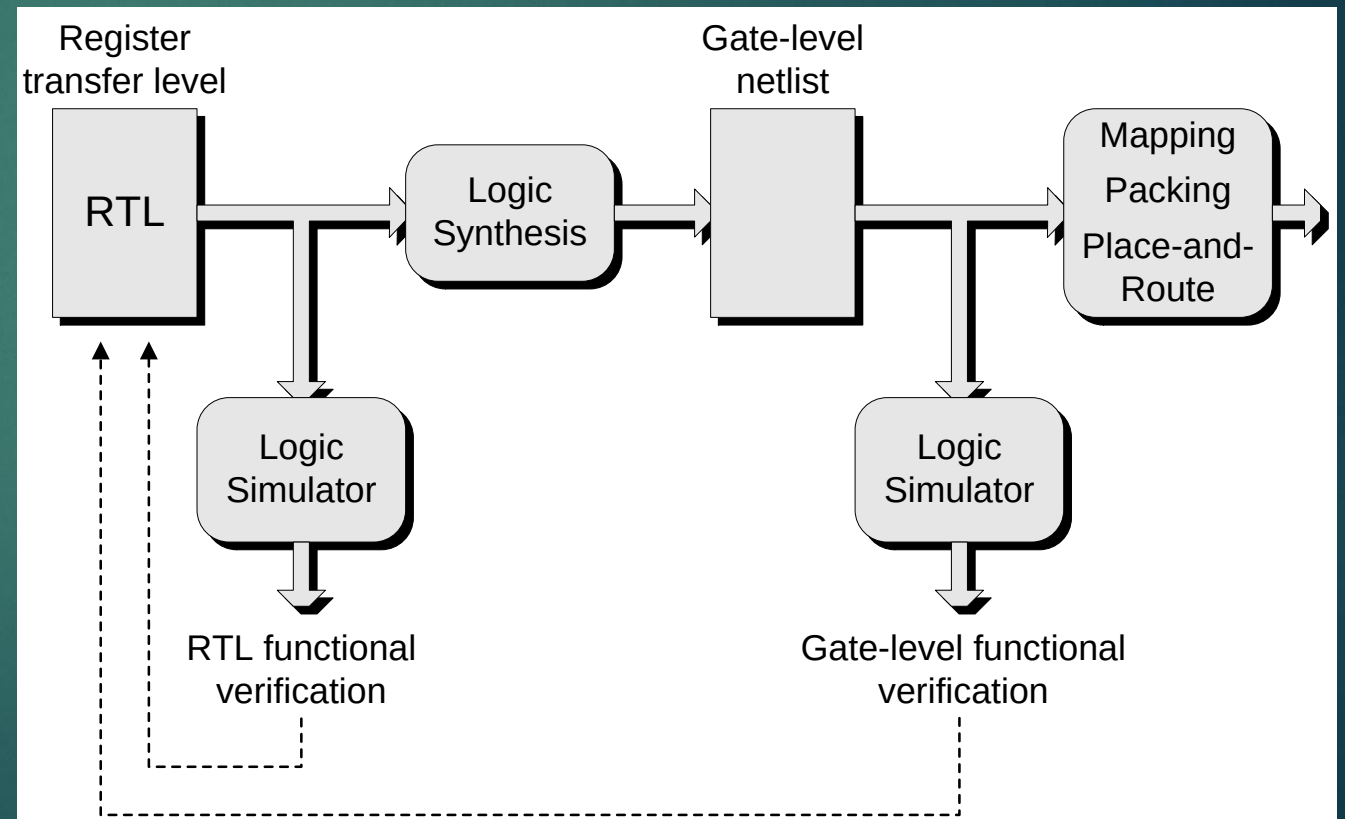




# Hardware Description Languages

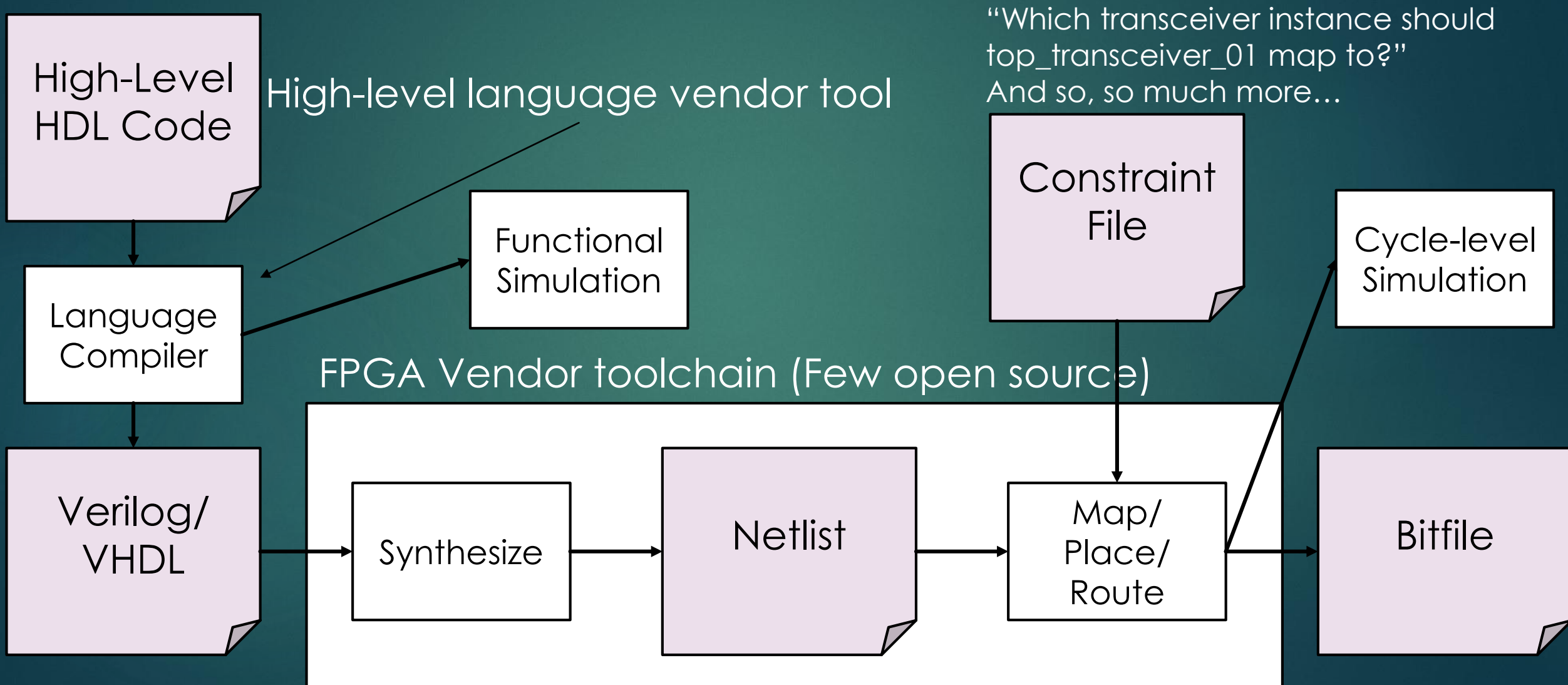
38

- ▶ Synthesis (Compilation)
- ▶ Generate Netlist



# FPGA Compilation Toolchain

39



# High-Level Synthesis (HLS)

- ▶ Compiler translates software programming languages to RTL
- ▶ High-Level Synthesis compiler from Xilinx, Altera/Intel
  - ▶ Compiles C/C++, annotated with **#pragma**'s into RTL
  - ▶ Theory/history behind it is a complex can of worms we won't go into
  - ▶ Personal experience: needs to be HEAVILY annotated to get performance
  - ▶ Anecdote: Naïve RISC-V in Vivado HLS achieves IPC of 0.0002 [1], 0.04 after optimizations [2]
- ▶ OpenCL
  - ▶ Inherently parallel language more efficiently translated to hardware
  - ▶ Stable software interface

[1] <http://msyksphinz.hatenablog.com/entry/2019/02/20/040000>

[2] <http://msyksphinz.hatenablog.com/entry/2019/02/27/040000>

# HLS Execution Control

41

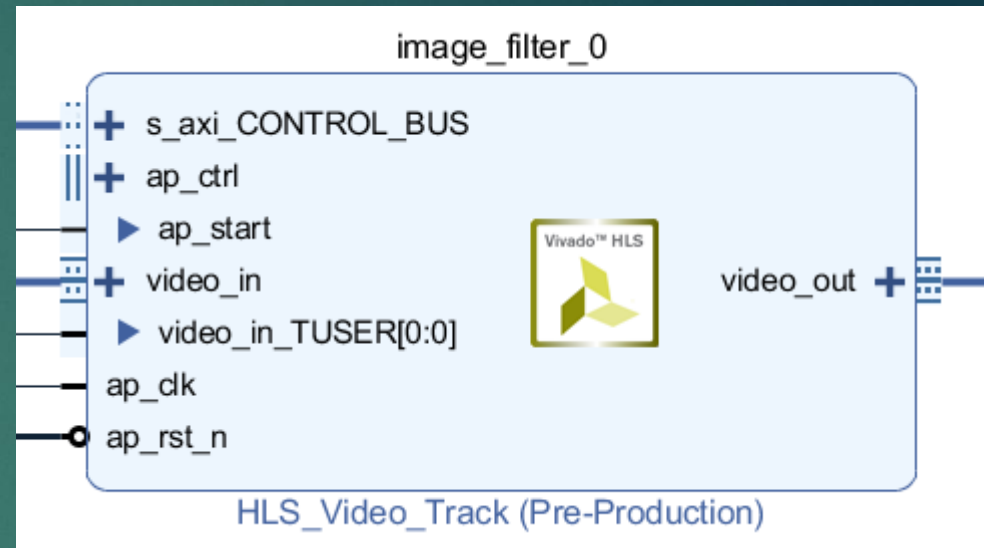
C++ code

```
void sgemv(  
    float *A,  
    float *x,  
    float *y,  
    float a,  
    float b)  
{  
    for (int i = 0; i < n; i++) {  
        float s = 0;  
        for (int j = 0; j < m; j++) {  
            s += A[i*m+j]*x[j];  
        }  
        y[i] = a*s+b*y[i];  
    }  
}
```

HLS

VHDL code

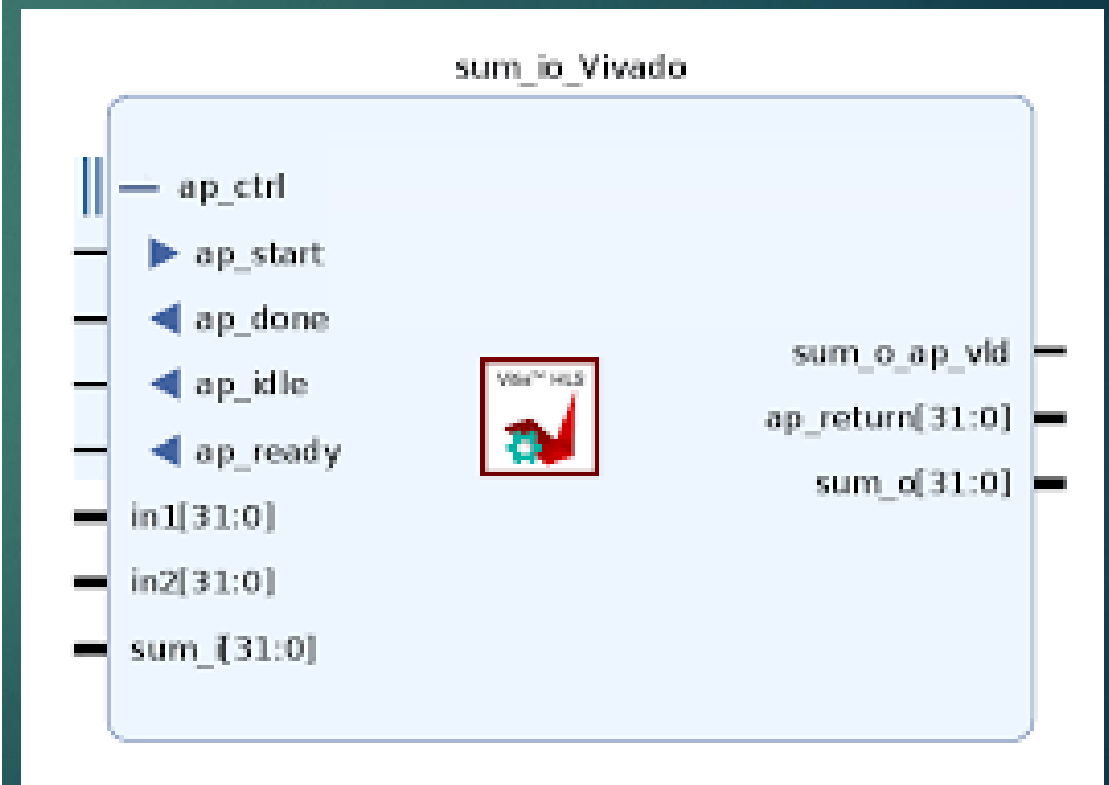
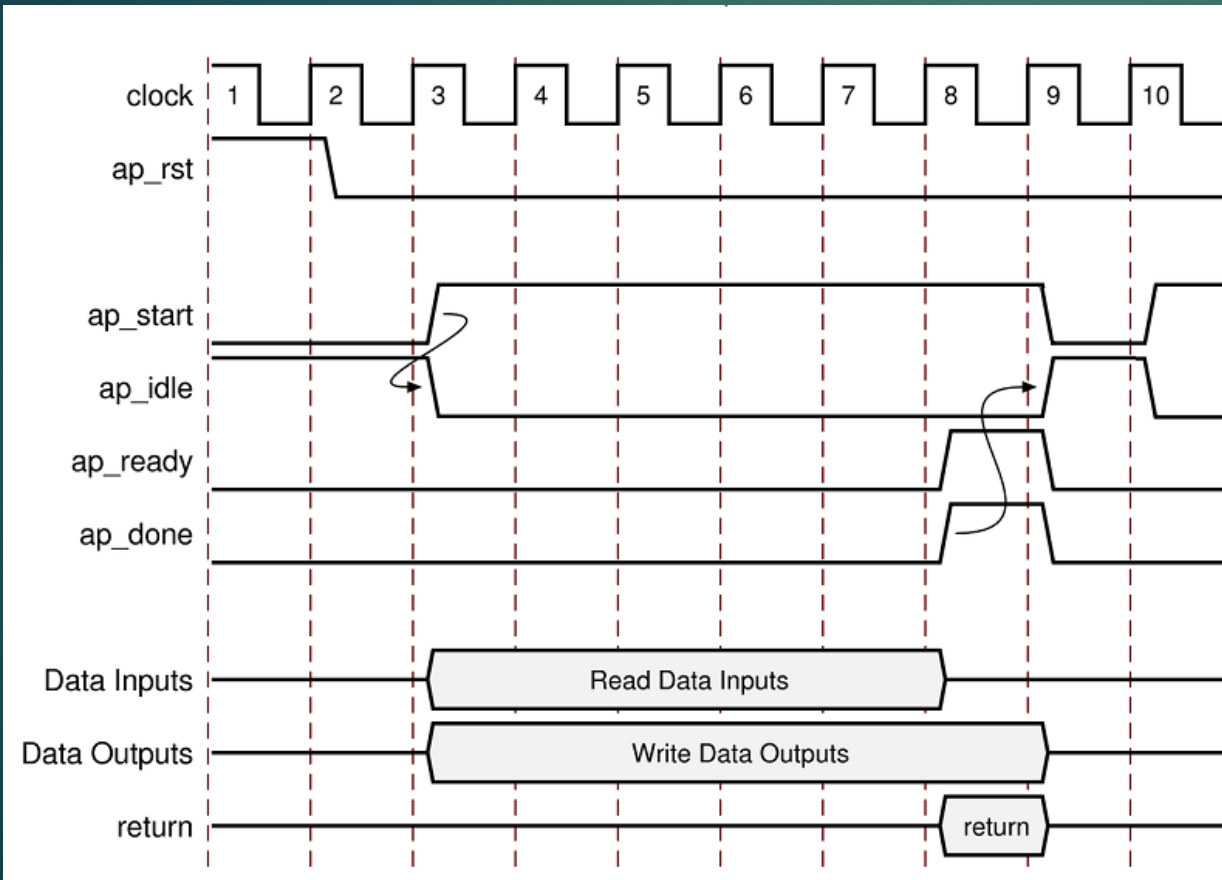
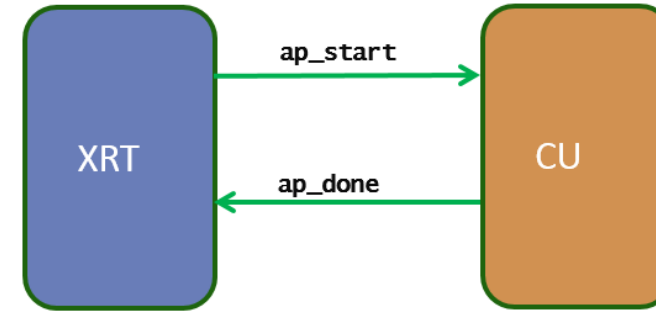
```
library ieee;  
use ieee.std_logic_1164.all;  
  
entity sgemv is  
    Port(...);  
end sgemv;  
  
architecture fpga of sgemv is  
    Signal ...;  
begin  
    ...  
end fpga;
```





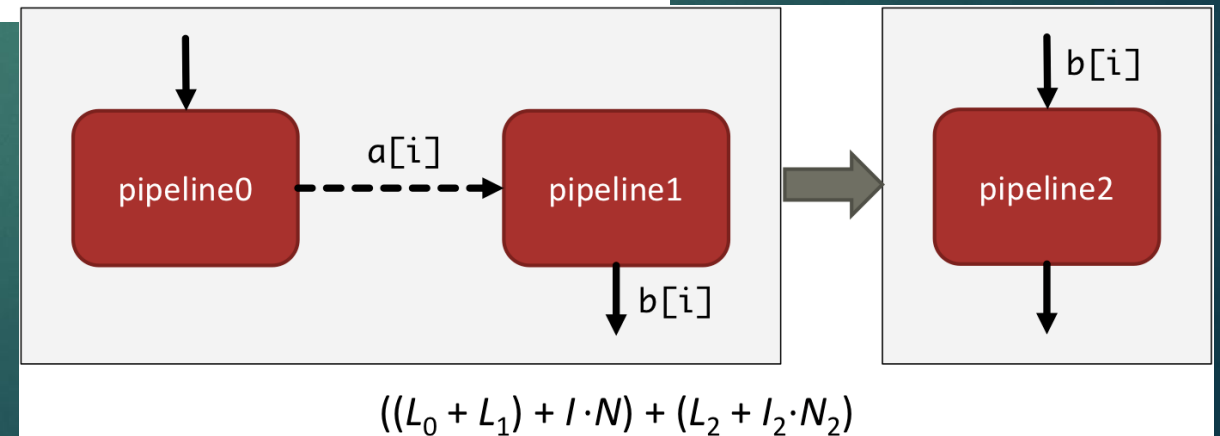
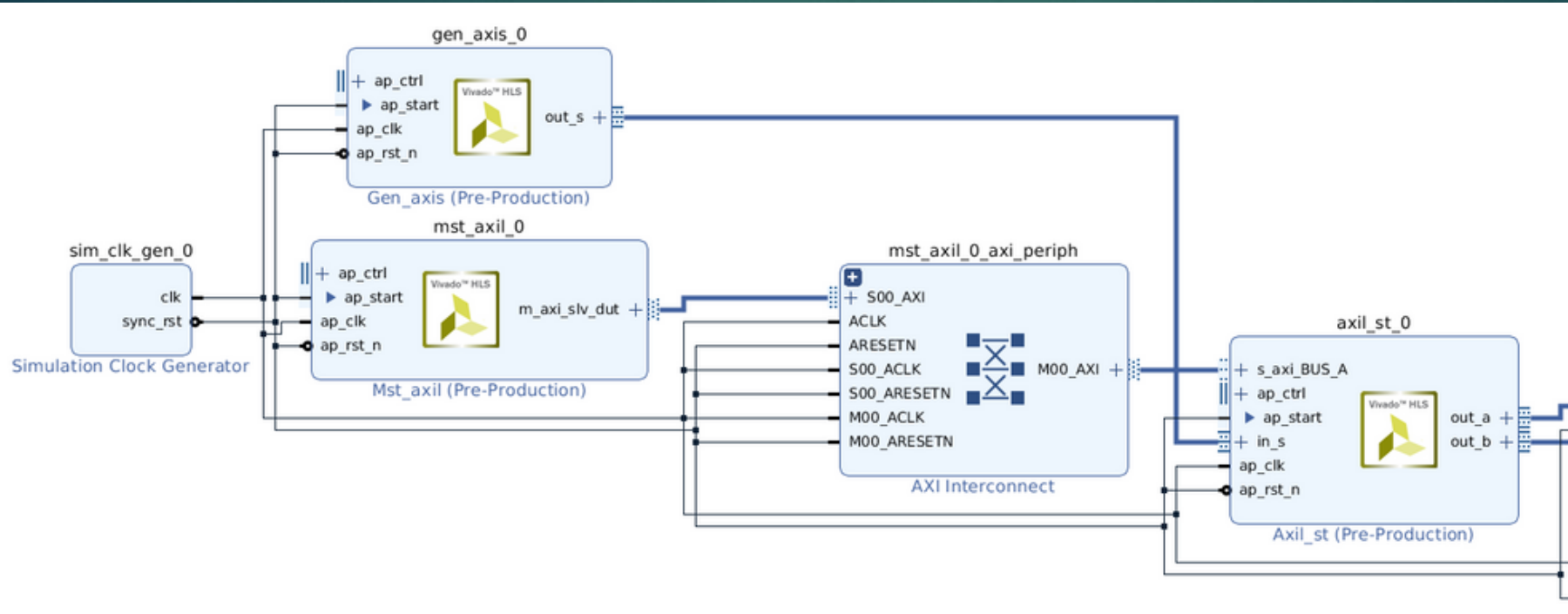
# HLS Execution Signals

42



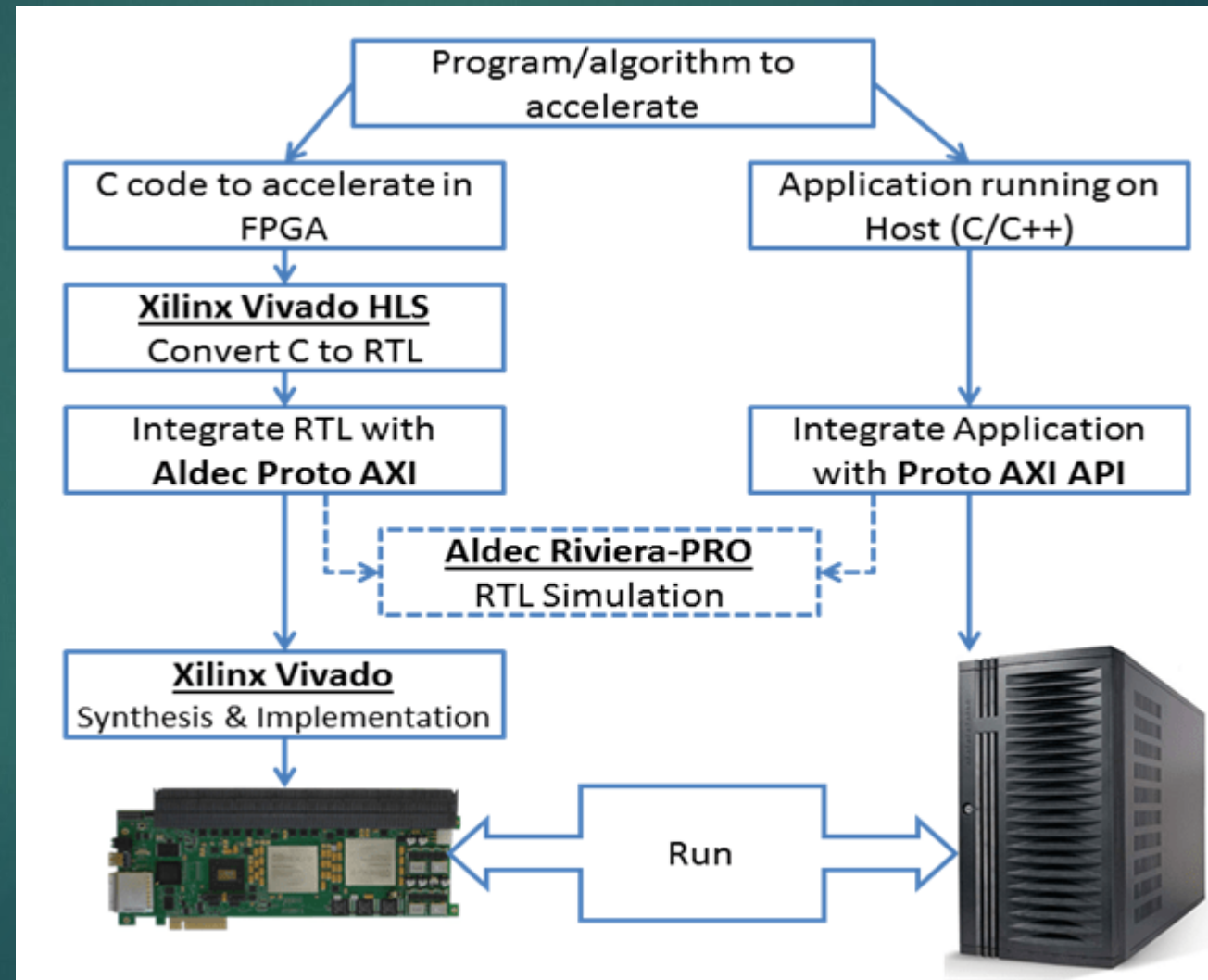
# HLS Execution in Digital Circuits

43



# HLS Design Flows

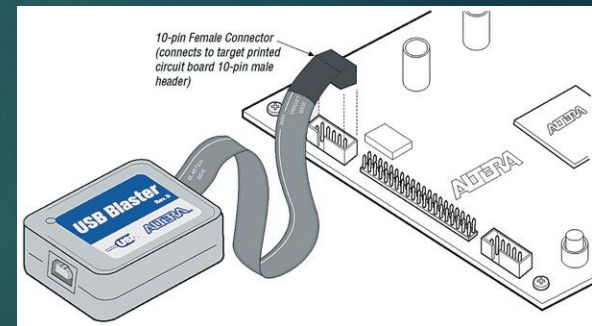
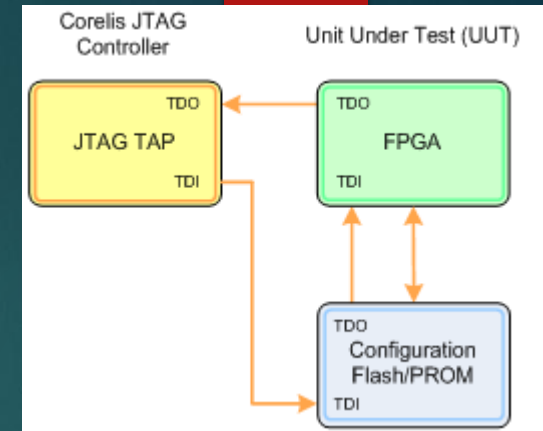
44



# Programming/Using an FPGA Accelerator

45

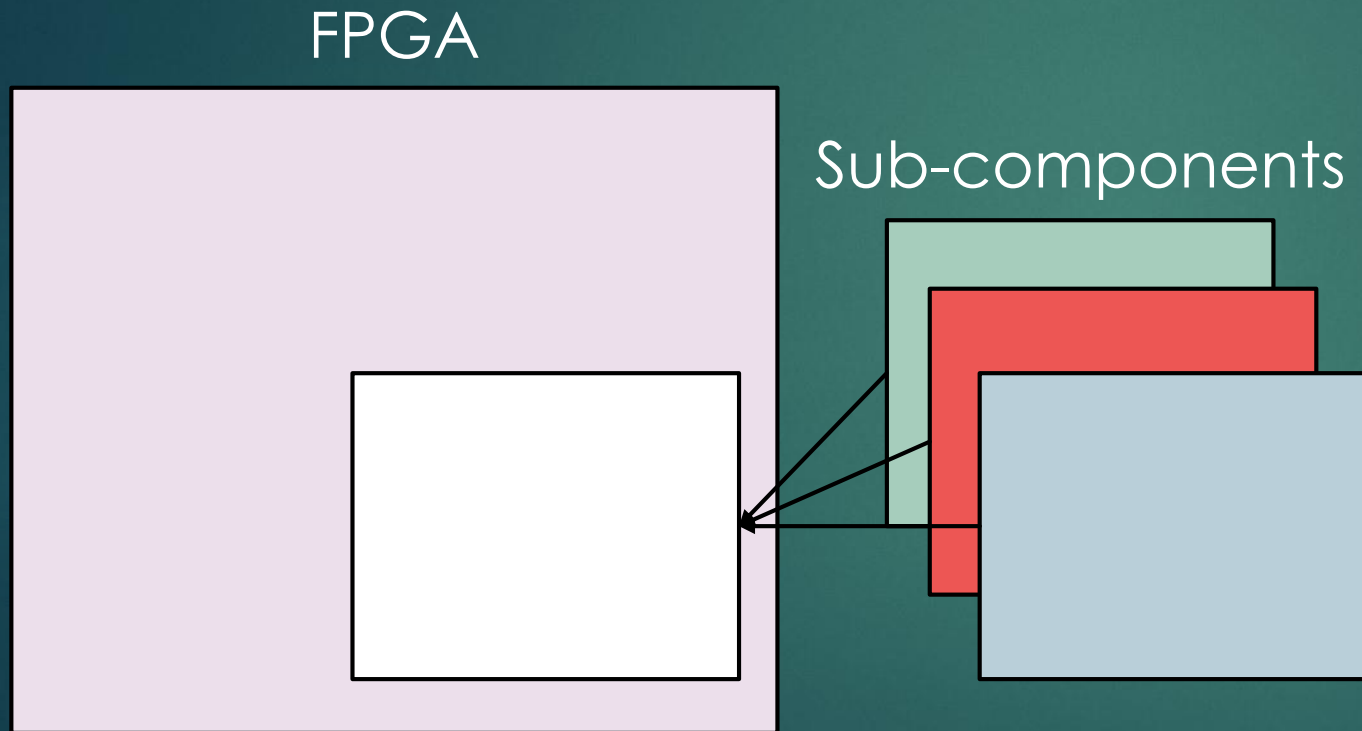
- ▶ Bitfile is programmed to FPGA over “JTAG” interface
  - ▶ Typically used over USB cable
  - ▶ Supports FPGA programming, limited debugging access, etc
- ▶ PCIe-attached FPGA accelerator card is typically used similarly to GPUs
  - ▶ Program FPGA, execute software
  - ▶ Software copies data to FPGA board, notify FPGA
    - > FPGA logic performs computations
    - > Software copies data back from FPGA
- ▶ FPGA flexibility gives immense freedom of usage patterns
  - ▶ Streaming, coherent memory, ...





# Partial Reconfiguration

46

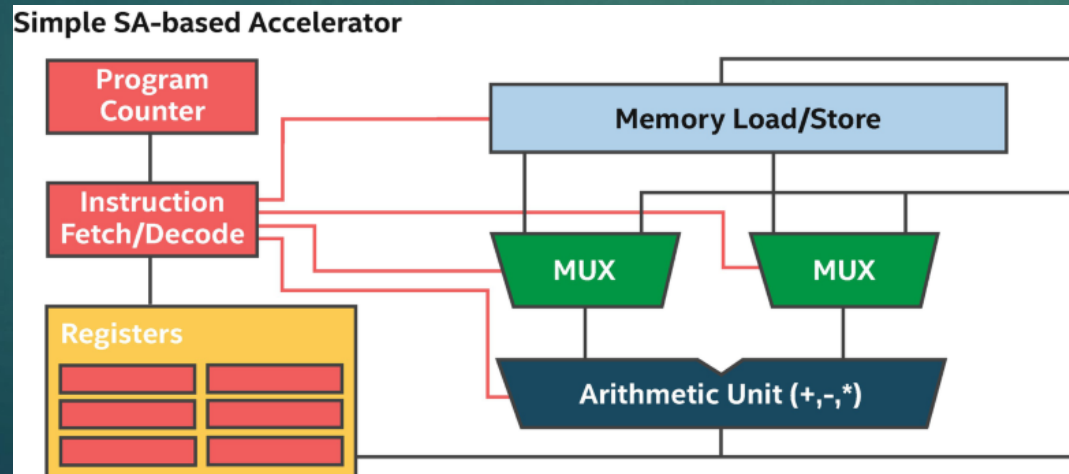
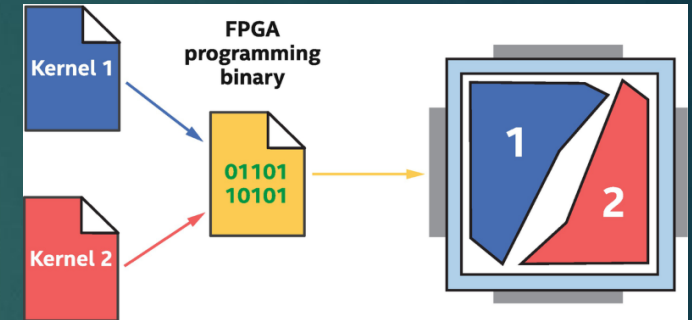


- ▶ Parts of the FPGA can be swapped out dynamically without turning off FPGA
  - ▶ Physical area is drawn on chip
- ▶ Used in Amazon F1, etc
- ▶ Toolchain support for isolation

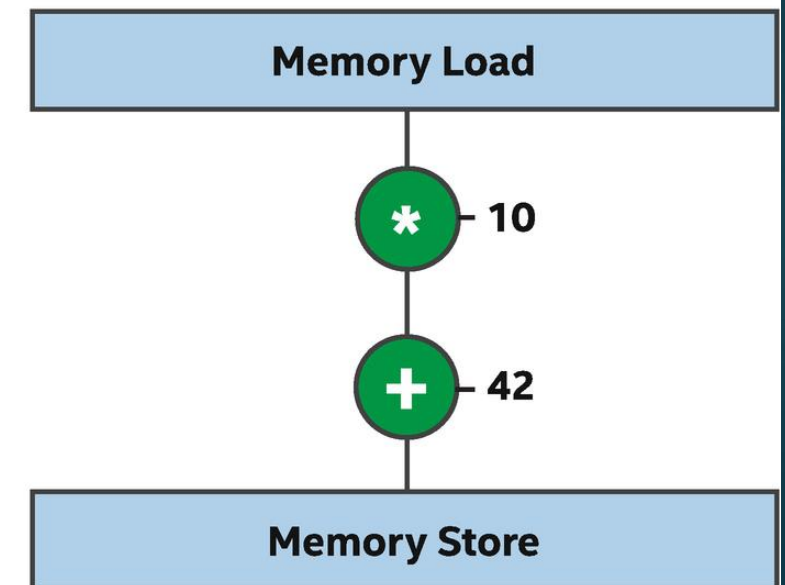
# Instructional vs Spatial Computing

47

- ▶ Processor-Based => Processing by Instructions
  - ▶ CPU -> Control and Compute
  - ▶ GPU -> Massively Compute
- ▶ Hardware-Based => Processing by Logic Gates
  - ▶ FPGA -> Programmable
  - ▶ ASIC -> Fixed



## Simple Spatial Compute Machine

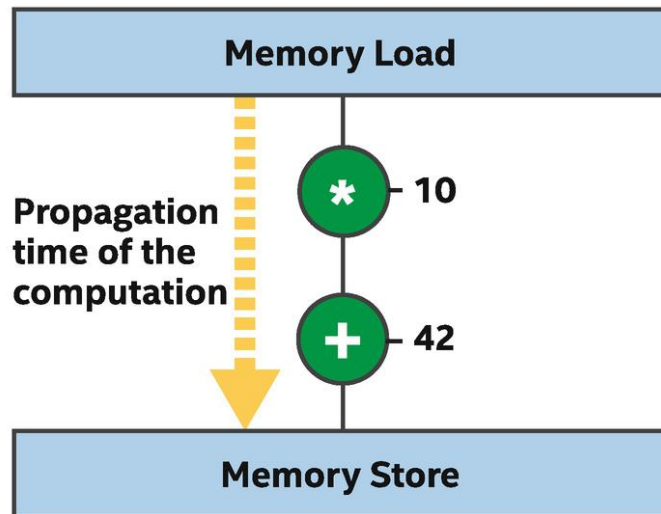


# Pipeline in FPGA Computing

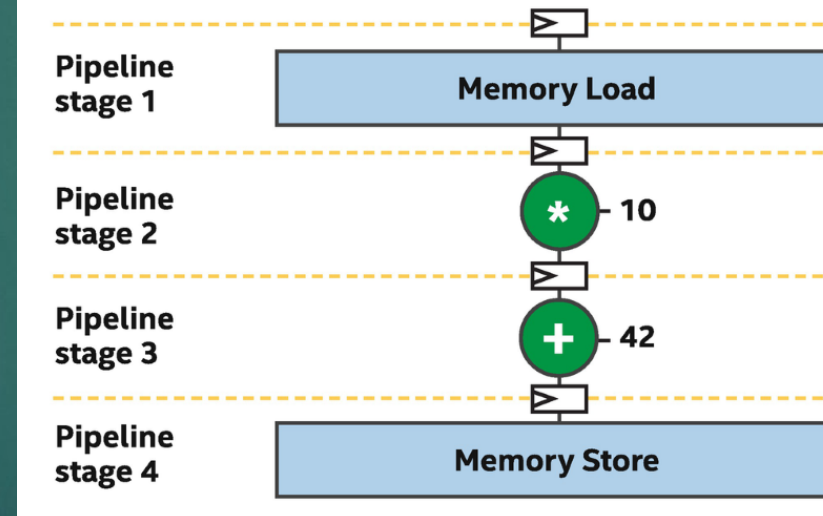
48

- More Parallel Computing using more parallel logic gates

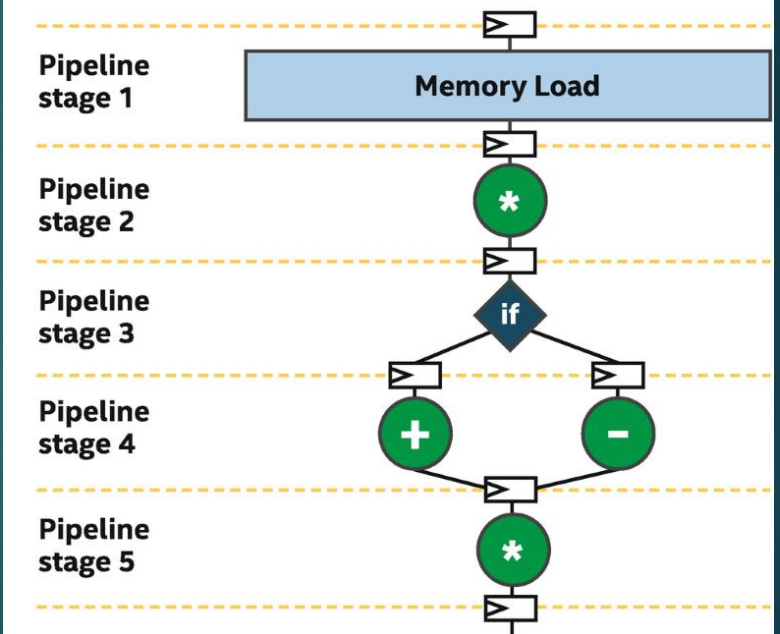
Spatial Compute



Pipelined Spatial Compute



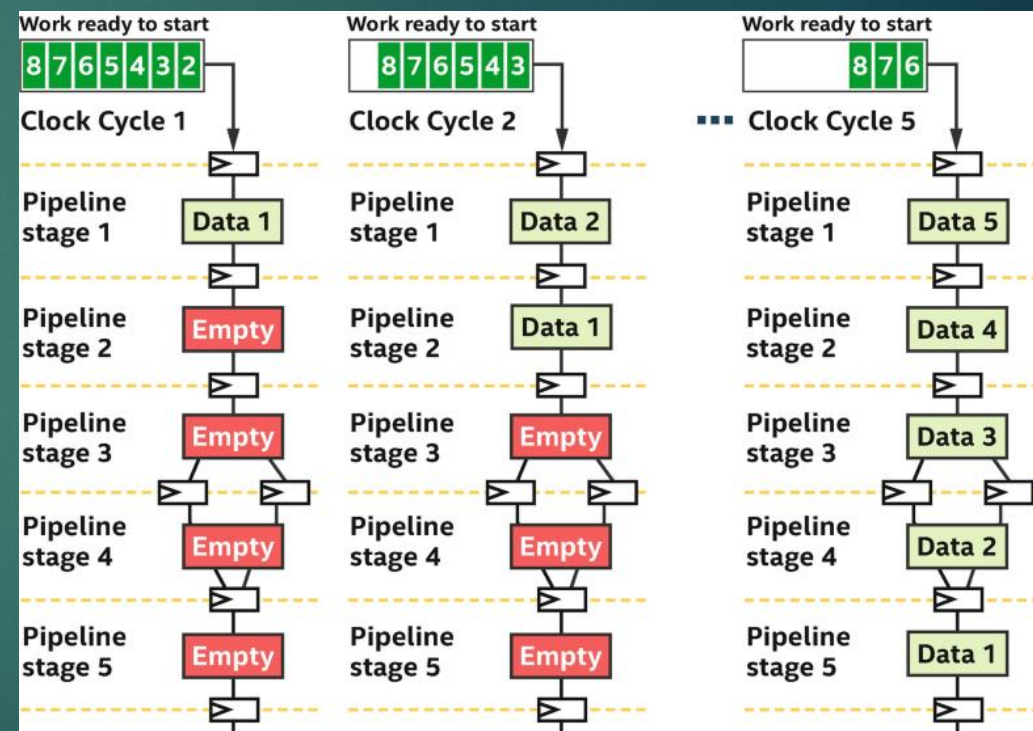
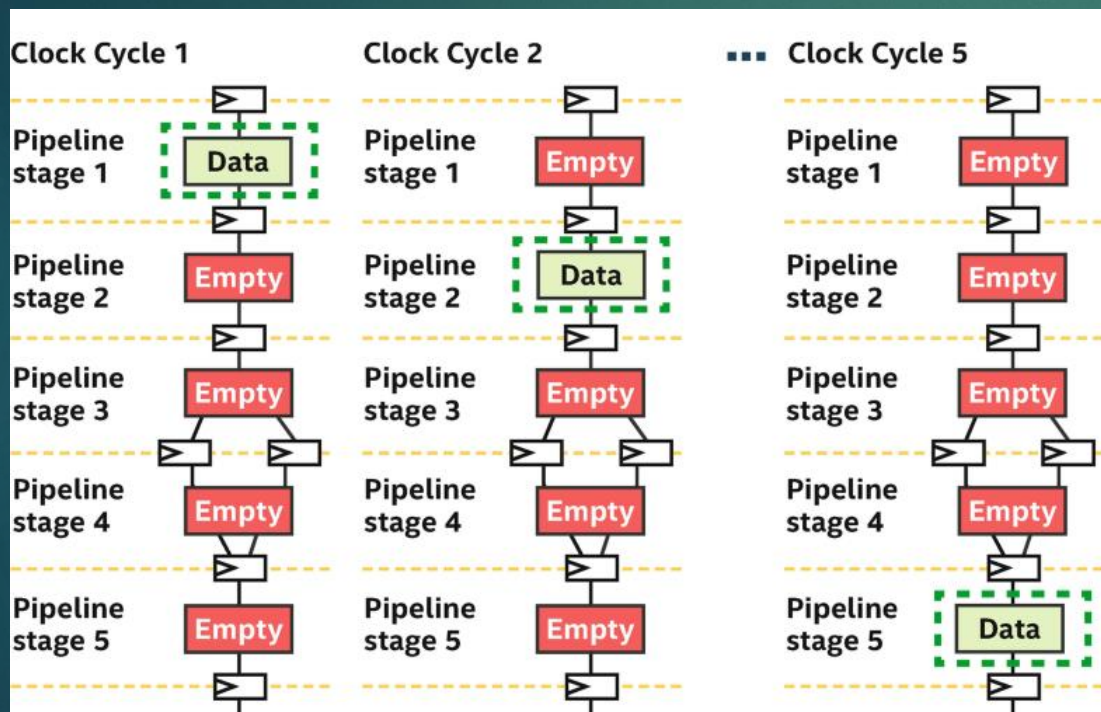
Pipelined Spatial Compute



# Pipeline in FPGA Computing

49

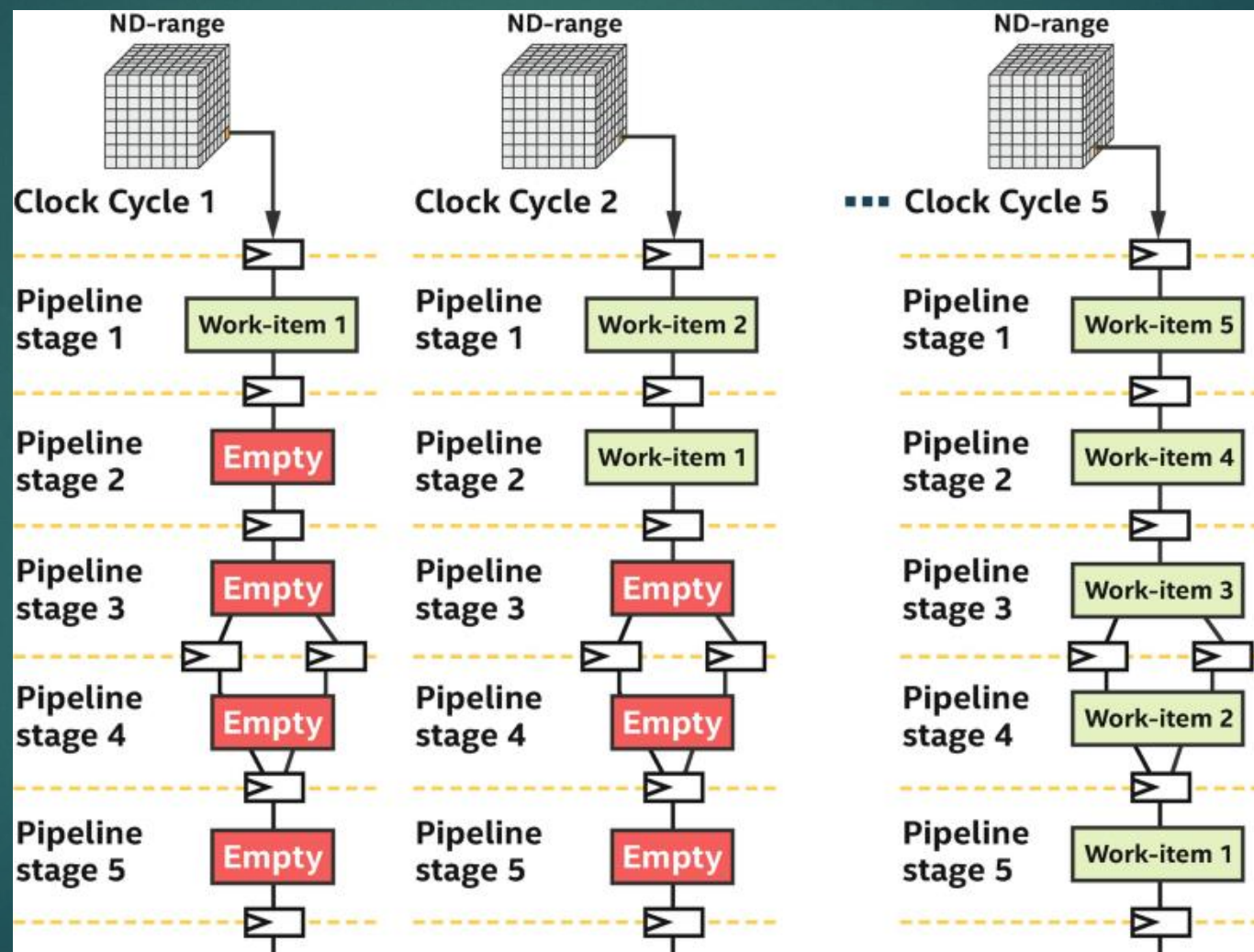
## ► Parallel Computing in multiple stages





# Pipeline for Work-Items

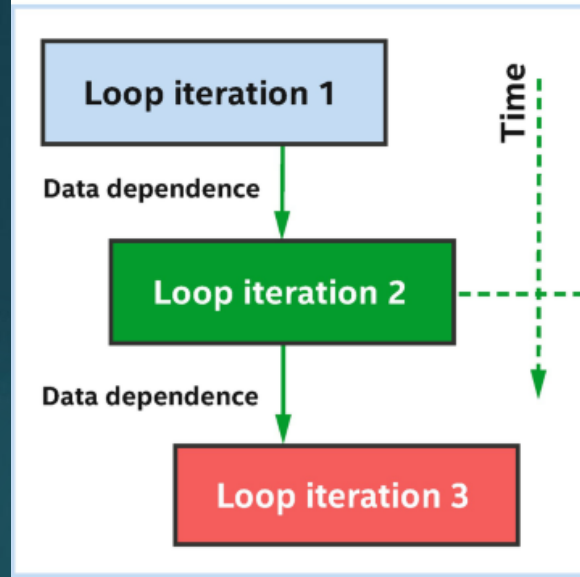
50



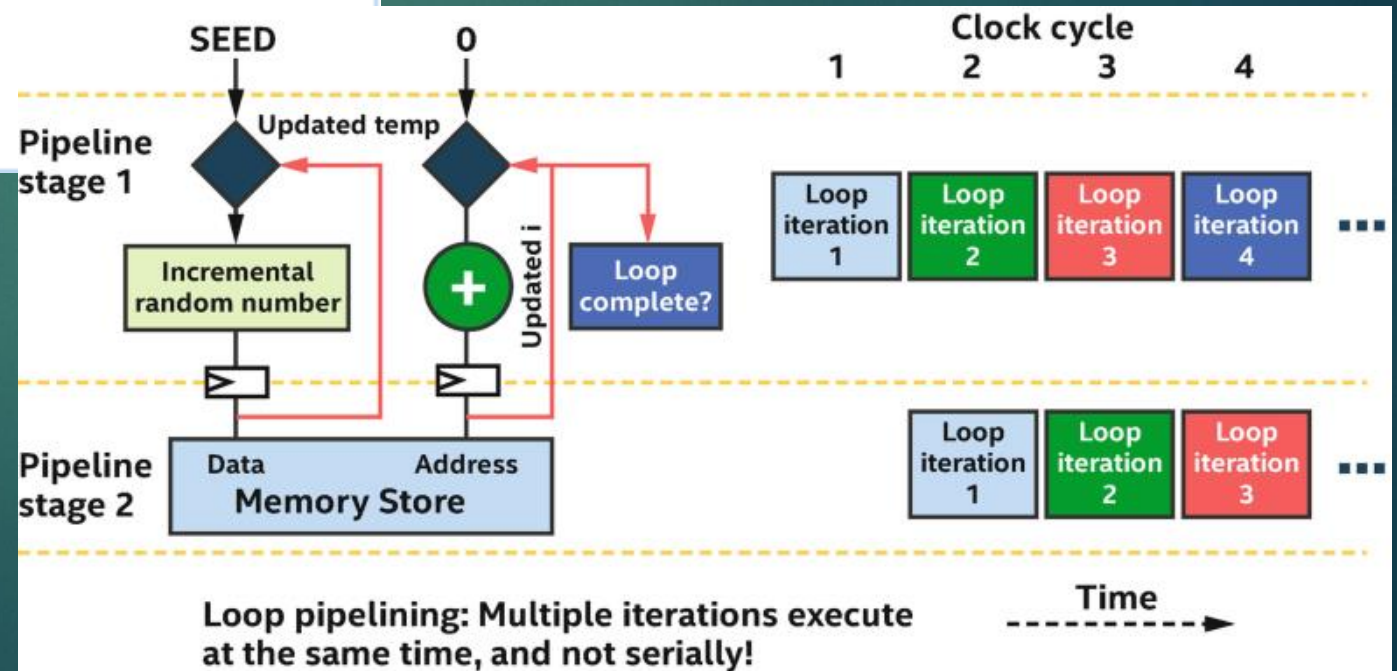
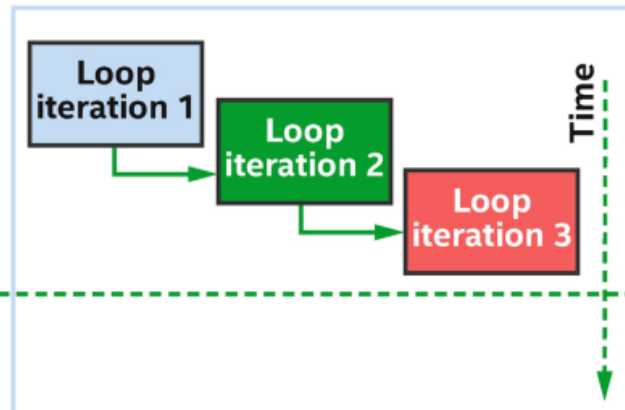
# Loop Pipeline

51

Serial execution of loop

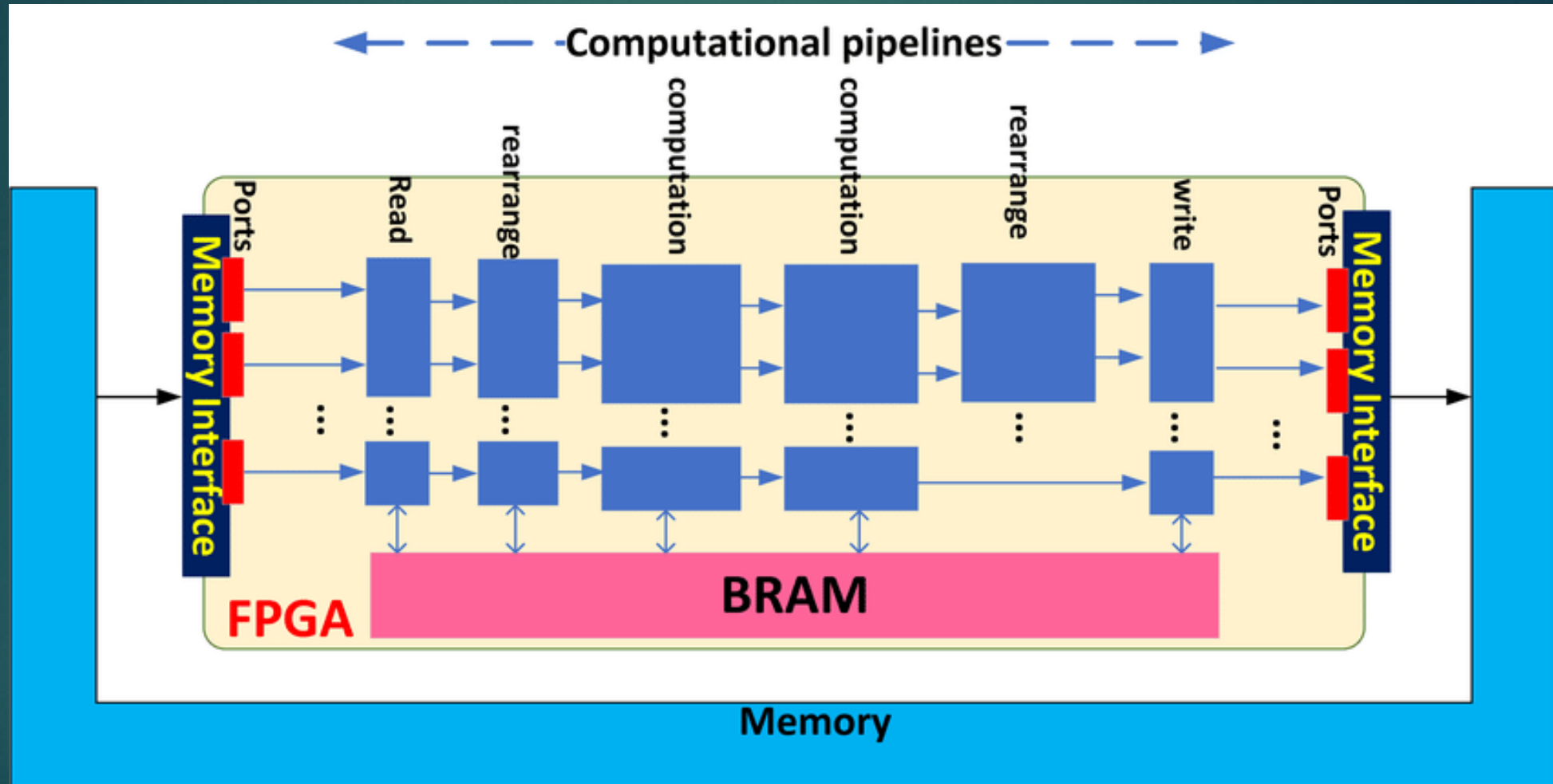


Loop pipelined execution



# FPGA Computation

52



# FPGAs In The Cloud

53

- ▶ Amazon EC2 F1 instance (1 – 4 FPGAs)
- ▶ Microsoft Azure, etc...

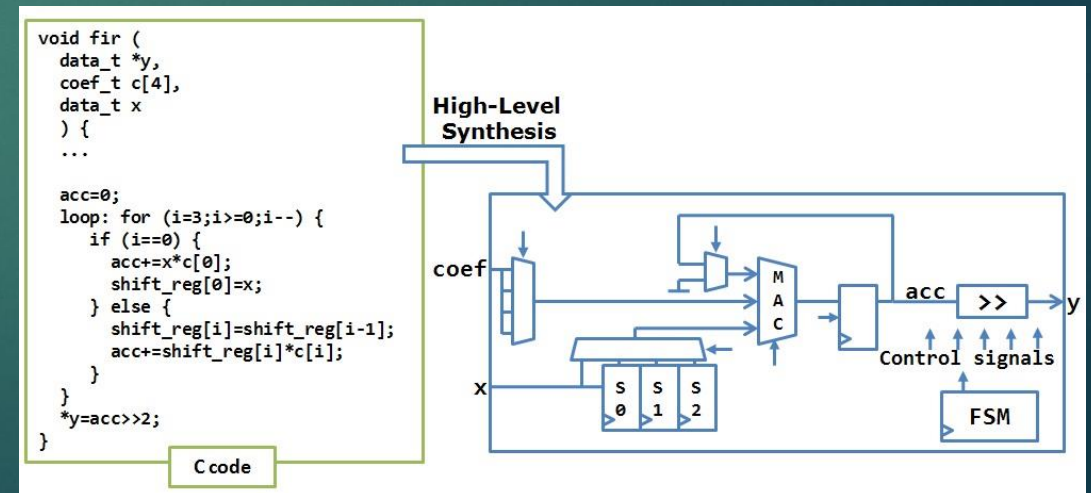
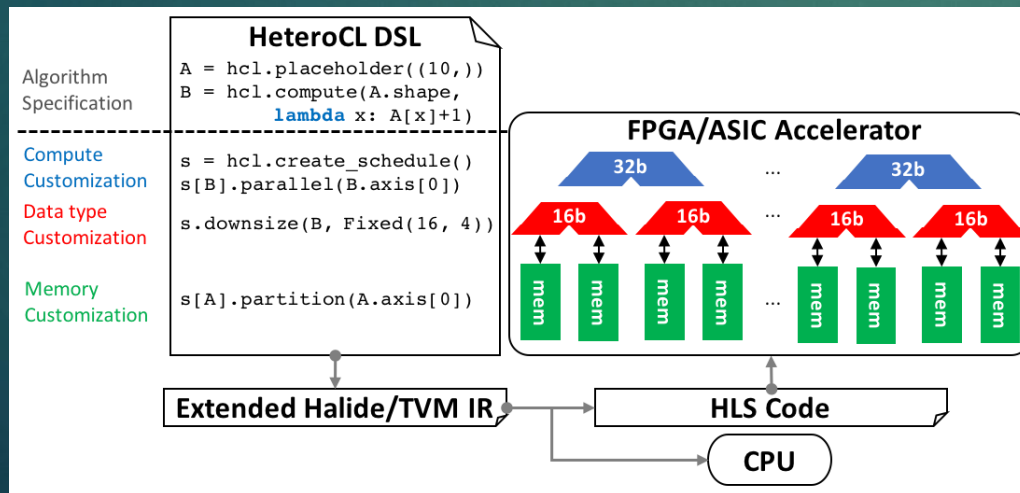
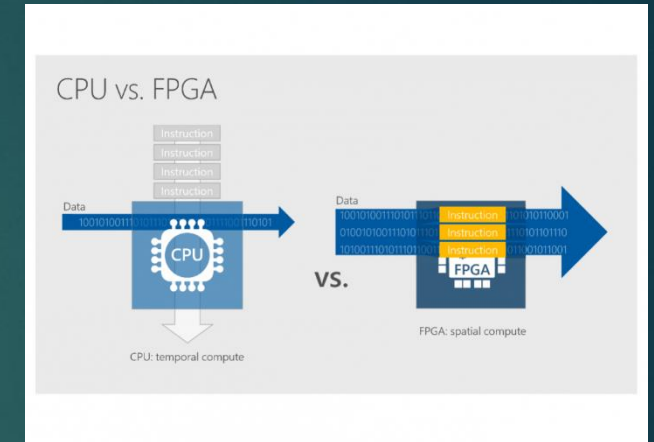




# Conclusion

54

- ▶ FPGA - **programmable** logic device
  - ▶ CLB, Switch, IO, Specific
  - ▶ Logic Gates as Functions
- ▶ **Source Code** be compiled by **HLS** to **HDL Code**
- ▶ HDL Code be compiled to **Configuration Code**
- ▶ Code run as **parallel logic gates directly**, under Host



# END

QUESTIONS & ANSWERS